

Assessment title: Assignment 1 – Server Side Application

Name: Abdur Rahim Sujon

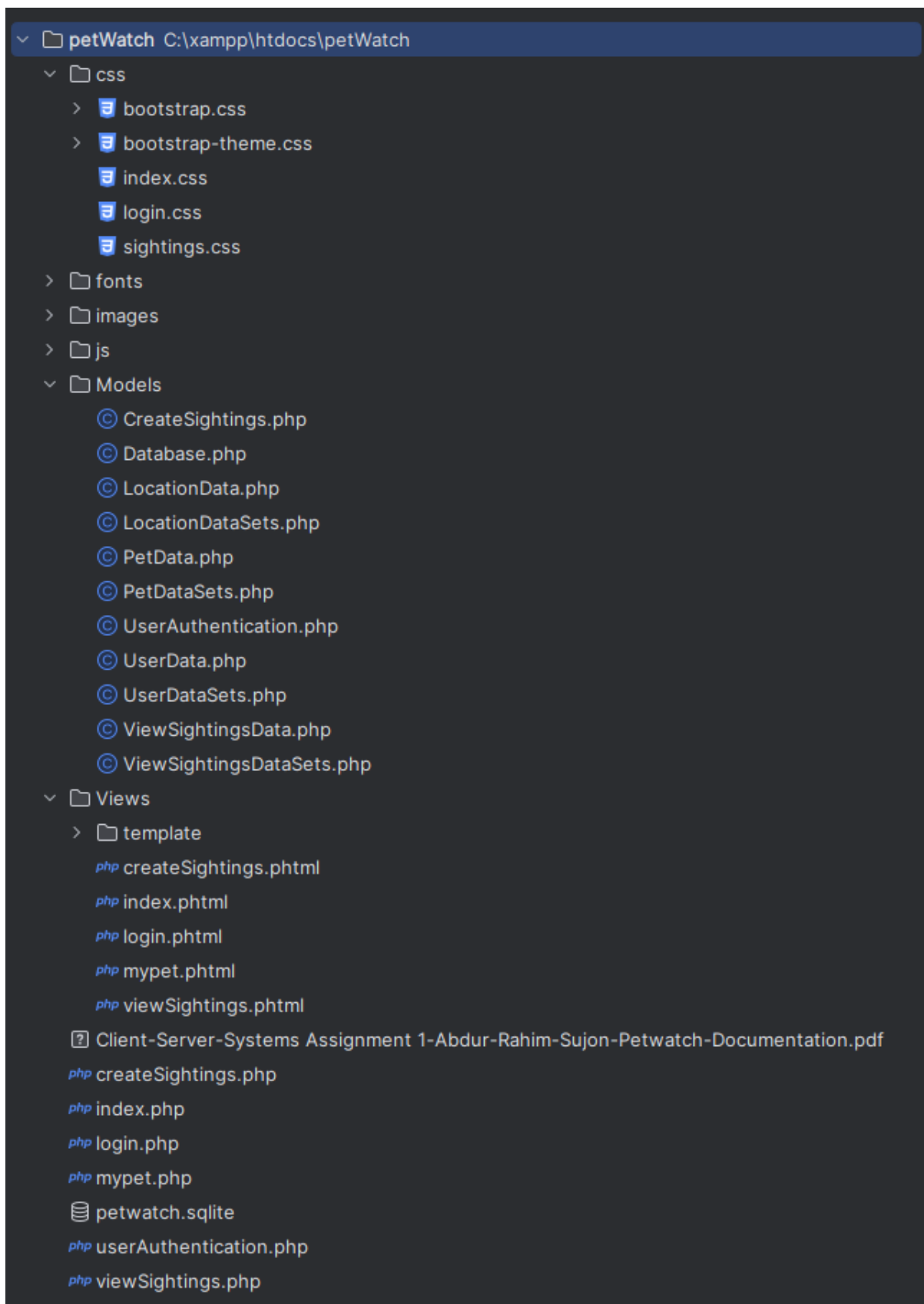
ID: @00768254

Assignment URL: <http://localhost:8000/petWatch>

User accounts (for testing/marking)

Type	Username	Password
Owner	Lee	O7m!eX#2vLp9@zFq
User	Zara	U\$4PzN!w8Hq@1rBx

A) Project File Structure



B) Self assessment of the marking scheme grid

Marking scheme Percentage Mark	Technical implementation (20 marks)	User Feature (20 marks)	Browsing User Display (20 marks)	Search feature (20 marks)	Create/Edit Records (20 marks)
Outstanding 90–100%	An outstanding implementation of OO code using the MVC pattern consistently and effectively. Inheritance, classes, properties, and methods are applied with precision and meaningful names. Demonstrates outstanding performance enhancements. Code is exceptionally well organised, excellently commented, and highly readable. Fully meets all technical requirements using SQLite via phpStorm/XAMPP/PHP CLI. Documentation is of outstanding quality.	An outstanding user feature set, including secure login with encrypted passwords, anti-spam, sessions, validation, and malicious code filtering. Driven by outstanding OO design and excellently commented code. Test accounts fully working.	An outstanding browsing feature, allowing retrieval and display of pet and sighting details with responsive CSS/Bootstrap, efficient paging, and large datasets. Driven by outstanding OO design and excellently commented code.	An outstanding interactive faceted search that reliably narrows output to fewer than 10 results using all record parameters. Implements complex SQL statements. Driven by excellent, logical OO design. Excellently commented code. Fully tested and fully functional.	An outstanding implementation of record management, allowing owners to add, view, and edit pet records, with browsing users able to leave sighting reports. Driven by outstanding OO design and excellently commented code.
Excellent 80–89%	An excellent implementation of OO code using the MVC pattern. Inheritance and class structures are well designed and appropriately applied. Includes some excellent performance enhancements. Code is well organised, consistently commented, and clearly readable. Meets technical requirements. Documentation is excellent.	An excellent user feature set, including secure login with password encryption, session management, and validation. Driven by excellent OO design and well-commented code. Test accounts working.	An excellent browsing feature, retrieving and displaying pet/sighting details with images. Includes a responsive layout and basic paging. Well-structured OO design, well commented.	An excellent free-text search facility using 3 parameters combined. Good OO implementation with well-commented code. Functional and reliable search with minor issues.	An excellent implementation of record management, with owners able to add/view records and browsing users able to leave reports. Good OO design and excellent commenting.
Very Good 70–79%	A very good implementation of OO code with strong adherence to MVC. Some inheritance and design are evident, though not always consistent. Code is generally well organised, readable, and well commented. Documentation is very good. Meets technical requirements with only minor issues.	A very good user feature set, with functional login, encryption, and session management. Some validation attempted. Driven by very good OO design and adequately commented code. Test accounts mostly working.	A very good browsing feature, retrieving item details with some formatting (tables/divs). Layout functional, with very good OO design and adequate comments.	A very good free-text search facility working but basic. Some filters implemented, minor issues present. Basic OO implementation. Some code comments included.	A very good implementation of record management. Users can add/view records and leave reports, though with some limitations. Adequate OO design and comments.

C) All 11 Model Files

1. Database.php

```
<?php

/**
 * Class Database designed to establish connection
 * to the database petwatch.sqlite
 */
class Database {
    /**
     * @var Database
     */
    protected static $_dbInstance = null;

    /**
     * @var PDO
     */
    protected $_dbHandle;

    /**
     * @return Database
     */
    public static function getInstance() {
        // Checks if the PDO exists
        if (self::$_dbInstance === null) {
            // Creates new instance if not, sending in connection info
            self::$_dbInstance = new self();
        }
        return self::$_dbInstance;
    }

    /**
     * Private constructor to initialize the database connection.
     * Uses SQLite and sets error handling to exception mode.
     * Terminates the script if the connection fails.
     */
    private function __construct() {
        try {
            $this->_dbHandle = new PDO("sqlite:petwatch.sqlite");
            // Handles error
            $this->_dbHandle->setAttribute(
                PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION
            );
        } catch (PDOException $e) {
            echo "Database Connection Error: " . $e->getMessage();
            // Stop script execution on connection failure
            die();
        }
    }

    /**
     * @return PDO
     */
    public function getdbConnection() {
        // Returns the PDO handle to be used elsewhere
        return $this->_dbHandle;
    }
}
```

```

/**
 * Destroys the PDO handle when no longer needed
 */
public function __destruct() {
    $this->_dbHandle = null;
}
}

```

2. UserData.php

```

<?php

/**
 * Class UserData is designed to initialise get functions
 * to retrieve user data.
 *
 * It works as a data model for UserDataSet class.
 */
class UserData {
    /**
     * @var int $_id User's unique identifier.
     * @var string $_username User's username.
     * @var string $_email User's email address.
     * @var string $_password_hash User's hashed password.
     * @var string $_role User's role (Owner/User).
     */
    protected $_id, $_username, $_email,
                $_password_hash, $_role;

    // Constructor of class UserData
    public function __construct($dbRow) {
        $this->_id = $dbRow['id'];
        $this->_username = $dbRow['username'];
        $this->_email = $dbRow['email'];
        $this->_password_hash = $dbRow['password_hash'];
        $this->_role = $dbRow['role'];
    }

    // Get user id
    public function getUserId() {
        return $this->_id;
    }

    // Get username
    public function getUsername() {
        return $this->_username;
    }

    // Get user email address
    public function getEmail() {
        return $this->_email;
    }

    // Get password hashed in the database
    public function getPasswordHash() {
        return $this->_password_hash;
    }

    // Get user role (Owner/User)

```

```

public function getRole() {
    return $this->_role;
}
}

```

3. UserDataSets.php

```

<?php
// Inheritance Database and UserData class models
require_once('Database.php');
require_once('UserData.php');

/**
 * Class UserDataSet
 * Handles interactions with the users table in the
 * database. Responsible for fetching user data
 * and returning it as UserData objects.
 */
class UserDataSet {
    /**
     * @var PDO $_dbHandle PDO database connection handle.
     * @var Database $_dbInstance Singleton instance
     * of the Database class.
     */
    protected $_dbHandle, $_dbInstance;

    /**
     * UserDataSet constructor.
     * Initializes database connection using the
     * Database singleton.
     */
    public function __construct() {
        $this->_dbInstance = Database::getInstance();
        $this->_dbHandle =
            $this->_dbInstance->getDbConnection();
    }

    /**
     * Retrieve a single user record by username.
     * Used to check if a username already exists
     * (e.g., during registration).
     */
    public function getUserByUsername($username) {
        // Check username existence, limit to one
        $sqlQuery = "SELECT * FROM users
                     WHERE username = :username
                     LIMIT 1";
        $statement = $this->_dbHandle->prepare($sqlQuery);
        $statement->bindParam(':username', $username);
        $statement->execute();

        // Fetch row and return as UserData object
        $row = $statement->fetch();
        if ($row) {
            return new UserData($row);
        }
        return null;
    }
}

```

4. UserAuthentication.php

```
<?php
// Inheritance the database connection using this two models
require_once('Database.php');
require_once('UserData.php');

// This class is used to authenticate user
// and owner with password hash functionality
class UserAuthentication {
    private $_dbHandle;

    // Get the database connection by inheritance
    public function __construct() {
        $this->_dbHandle =
            Database::getInstance()->getDbConnection();
    }

    /**
     * Authenticate a user by username and password.
     * Returns a UserData object if successful, null otherwise.
     */
    public function verifyLogin($username, $password): ?UserData {
        // Select user record by username (limit to one)
        $sql = "SELECT * FROM users
                WHERE username = :username
                LIMIT 1";
        $stmt = $this->_dbHandle->prepare($sql);
        $stmt->bindValue(':username', $username);
        $stmt->execute();

        // Fetch the user data as an associative array
        $row = $stmt->fetch(PDO::FETCH_ASSOC);

        // Verify the password against hashed password in db
        if ($row && password_verify($password,
            $row['password_hash'])) {
            // Return the matched user details
            return new UserData($row);
        }

        // If credentials incorrect, return null
        return null;
    }

    /**
     * Updates hashed password in the database.
     * Each time called, rehashes all user passwords.
     * Implemented by userAuthentication.php controller.
     */
    public function hasUserPassword(
        $ownerPassword, $userPassword
    ): int {
        // Select all user ids and roles
        $sql = "SELECT id, role FROM users";
        $stmt = $this->_dbHandle->prepare($sql);
        $stmt->execute();
        $users = $stmt->fetchAll(PDO::FETCH_ASSOC);
```

```

// Prepare update query for hashed password
$update = $this->_dbHandle->prepare("
    UPDATE users SET password_hash = :hash WHERE id = :id
");

$count = 0;

// Hash password for each user
foreach ($users as $user) {
    // Hash for owner or normal user accordingly
    $hash = password_hash(
        $user['role'] === 'Owner'
        ? $ownerPassword : $userPassword,
        PASSWORD_DEFAULT
    );

    // Update hashed password
    $update->execute([
        ':hash' => $hash,
        ':id' => $user['id']
    ]);

    // Increment total updated count
    $count += $update->rowCount();
}

// Return how many users were updated
return $count;
}
}

```

5. PetData.php

```

<?php

// Class model works as data model for PetDataSets.php class
class PetData {
    /**
     * @var int $_id pet id
     * @var string $_name pet name
     * @var string $_species pet type
     * @var string $_breed breed of the pet
     * @var string $_color color of the pet
     * @var string $_description how the pet looks and its traits
     * @var string $_pet_images pet pictures
     * @var string $_status lost or found
     * @var string $_date_reported date pet was reported
     * @var int $_user_id the user who owns the pet
     */
    protected $_id, $_name, $_species, $_breed, $_color,
        $_description, $_pet_images, $_status,
        $_date_reported, $_user_id;

    // Constructor
    public function __construct($dbRow) {
        $this->_id = $dbRow['id'];
        $this->_name = $dbRow['name'];
        $this->_species = $dbRow['species'];
    }
}

```

```

        $this->_breed = $dbRow['breed'];
        $this->_color = $dbRow['color'];
        $this->_description = $dbRow['description'];
        $this->_pet_images = $dbRow['photo_url'];
        $this->_status = $dbRow['status'];
        $this->_date_reported = $dbRow['date_reported'];
        $this->_user_id = $dbRow['user_id'];
    }

    // Get pet id
    public function getId() {
        return $this->_id;
    }

    // Get pet name
    public function getName() {
        return $this->_name;
    }

    // Get pet species (e.g. dog, cat, bird)
    public function getSpecies() {
        return $this->_species;
    }

    // Get the pet breed
    public function getBreed() {
        return $this->_breed;
    }

    // Get the pet color
    public function getColor() {
        return $this->_color;
    }

    // Get pet description
    public function getDescription() {
        return $this->_description;
    }

    // Get the pet image
    public function getPetImages() {
        return $this->_pet_images;
    }

    // Get the pet status (lost/found)
    public function getStatus() {
        return $this->_status;
    }

    // Get reported time that pet was lost
    public function getDateReported() {
        return $this->_date_reported;
    }

    // Get pet by user id
    public function getPetsByUser() {
        return $this->_user_id;
    }
}

```

6. PetDataSets.php

```
<?php
// Inheritance Database and PetData class
require_once('Database.php');
require_once('PetData.php');

// Class to establish connection between database and controller
// to get pets data
class PetDataSets {
    public $_dbHandle;
    protected $_dbInstance;

    // Inheritance database connection
    public function __construct() {
        $this->_dbInstance = Database::getInstance();
        $this->_dbHandle = $this->_dbInstance->getDbConnection();
    }

    // Get all pets belonging to a user
    public function getPetByUserID($user_id): array {
        // Select all pets for logged-in user by their user id
        // and show the newest first
        $sql = "SELECT * FROM pets
                WHERE user_id = :user_id
                ORDER BY date_reported DESC";
        $stmt = $this->_dbHandle->prepare($sql);
        // Bind user id value as integer
        $stmt->bindValue(':user_id', $user_id, PDO::PARAM_INT);
        $stmt->execute();

        // Fetch pets for specific logged-in user
        $pets = [];
        while ($row = $stmt->fetch(PDO::FETCH_ASSOC)) {
            $pets[] = new PetData($row);
        }
        return $pets; // Return fetched pets
    }

    // Create a new pet entry. Status always defaults to 'lost'
    public function addPet(
        $user_id, $name, $species, $breed,
        $color, $description, $photo_url = null
    ) {
        // Insert new data to database when user adds a new pet
        // Status = lost, Date = current datetime
        $sql = "INSERT INTO pets
                (user_id, name, species, breed, color,
                 description, photo_url, status, date_reported)
                VALUES
                (:user_id, :name, :species, :breed, :color,
                 :description, :photo_url, 'lost', datetime('now'))";

        $stmt = $this->_dbHandle->prepare($sql);
        // Safely bind values
        $stmt->bindValue(':user_id', $user_id, PDO::PARAM_INT);
        $stmt->bindValue(':name', $name);
        $stmt->bindValue(':species', $species);
        $stmt->bindValue(':breed', $breed);
        $stmt->bindValue(':color', $color);
        $stmt->bindValue(':description', $description);
```

```

$stmt->bindValue(':photo_url', $photo_url);
$stmt->execute();

// Return last inserted id for confirmation
return $this->_dbHandle->lastInsertId();
}

// Update a pet entry (only owner can update details)
public function updatePet($petId, $user_id, $data): bool {
    $fields = [];
    $params = ['id' => $petId, 'user_id' => $user_id];

    // Collect valid fields
    foreach ($data as $key => $value) {
        if (in_array($key, [
            'name', 'species', 'breed',
            'color', 'description', 'photo_url', 'status'
        ])) {
            $fields[] = "$key = :$key";
            $params[":$key"] = $value;
        }
    }

    // If owner has no pets, nothing to update
    if (empty($fields)) return false;

    // Join all field assignments for SQL update
    $sql = "UPDATE pets
        SET " . implode(', ', $fields) . "
        WHERE id = :id AND user_id = :user_id";
    $stmt = $this->_dbHandle->prepare($sql);
    $stmt->execute($params);

    // True if rows affected
    return $stmt->rowCount() > 0;
}

// Delete a pet (owner only)
public function deletePet($petId, $user_id): bool {
    // Check pet id and user id to ensure ownership
    $sql = "DELETE FROM pets
        WHERE id = :id AND user_id = :user_id";
    $stmt = $this->_dbHandle->prepare($sql);
    // Bind securely
    $stmt->bindValue(':id', $petId, PDO::PARAM_INT);
    $stmt->bindValue(':user_id', $user_id, PDO::PARAM_INT);
    $stmt->execute();

    // True if delete succeeded
    return $stmt->rowCount() > 0;
}
}

```

7. ViewSightingsData.php

```

<?php

/**
 * Class designed to handle data models for

```

```

* ViewSightingsDataSets class.
*/
class sightingsData {
/**
 * @var string $_pet_name Pet name.
 * @var string $_pet_species Pet species (dog, cat, etc.).
 * @var string $_pet_status Current status (lost/found).
 * @var string $_pet_images Pet images.
 * @var string $_comment Comment about the pet.
 * @var float $_latitude Sighting latitude.
 * @var float $_longitude Sighting longitude.
 * @var string $_timestamp Date and time recorded.
 */
protected $_pet_name, $_pet_species, $_pet_status,
$_pet_images, $_comment, $_latitude,
$_longitude, $_timestamp;

/**
 * Constructor to initialize sighting data
 * from a database row.
 */
public function __construct($dbRow) {
    $this->_pet_name = $dbRow['name'];
    $this->_pet_species = $dbRow['species'];
    $this->_pet_status = $dbRow['status'];
    $this->_pet_images = $dbRow['photo_url'];
    $this->_comment = $dbRow['comment'];
    $this->_latitude = $dbRow['latitude'];
    $this->_longitude = $dbRow['longitude'];
    $this->_timestamp = $dbRow['timestamp'];
}

// Get pet name
public function getPetName() {
    return $this->_pet_name;
}

// Get pet species
public function getPetSpecies() {
    return $this->_pet_species;
}

// Get pet status (lost/found)
public function getPetStatus() {
    return $this->_pet_status;
}

// Get pet images
public function getPetImages() {
    return $this->_pet_images;
}

// Get sightings comment
public function getSightingsComment() {
    return $this->_comment;
}

// Get latitude of the sighting
public function getSightingsLatitude() {
    return $this->_latitude;
}
}

```

```

// Get longitude of the sighting
public function getSightingsLongitude() {
    return $this->_longitude;
}

// Get sighting timestamp
public function getSightingsTimestamp() {
    return $this->_timestamp;
}
}

```

8. ViewSightingsDataSets.php

```

<?php
// Inheritance database connection class and
// ViewSightings model
require_once('Database.php');
require_once('ViewSightingsData.php');

// Class to handle sightings data
class ViewSightingsDataSet {
    protected $_dbHandle, $_dbInstance;

    // Constructor to handle database inheritance
    public function __construct() {
        $this->_dbInstance = Database::getInstance();
        $this->_dbHandle =
            $this->_dbInstance->getdbConnection();
    }

    /**
     * Fetch sightings with search, filter,
     * sorting, and pagination.
     * @param array $params Keys: search, status,
     * species, sort_order, sort_date
     * @param int $limit Max records to return
     * @param int $offset Start index for records
     * @return array Array of SightingsData objects
     */
    public function fetchSightingsWithFilters(
        array $params, int $limit, int $offset
    ): array {
        /**
         * Query retrieves pet and sighting details.
         * INNER JOIN ensures only pets with
         * sightings are included.
         */
        $sqlQuery = "SELECT p.name, p.species, p.status,
                        p.photo_url, s.comment, s.latitude,
                        s.longitude, s.timestamp
                    FROM sightings s
                    INNER JOIN pets p ON s.pet_id = p.id
                    WHERE 1=1";

        $bindParam = [];

        // Apply search filter
        if (!empty($params['search'])) {

```

```

$sqlQuery .= " AND (p.name LIKE :search
                OR p.species LIKE :search
                OR p.status LIKE :search
                OR s.comment LIKE :search
                OR s.latitude LIKE :search
                OR s.longitude LIKE :search)";
$bindParam[':search'] =
    '%' . $params['search'] . '%';
}

// Status filter
if (!empty($params['status'])) {
    $sqlQuery .= " AND p.status = :status";
    $bindParam[':status'] = $params['status'];
}

// Species filter
if (!empty($params['species'])) {
    $sqlQuery .= " AND p.species = :species";
    $bindParam[':species'] = $params['species'];
}

// Sort filter (date or name)
if (!empty($params['sort_date'])) {
    $sqlQuery .= $params['sort_date'] === 'oldest'
        ? " ORDER BY s.timestamp ASC"
        : " ORDER BY s.timestamp DESC";
} elseif (!empty($params['sort_order'])) {
    if ($params['sort_order'] === 'name_asc') {
        $sqlQuery .= " ORDER BY p.name ASC";
    } elseif ($params['sort_order'] === 'name_desc') {
        $sqlQuery .= " ORDER BY p.name DESC";
    } else {
        $sqlQuery .= " ORDER BY s.timestamp DESC";
    }
} else {
    $sqlQuery .= " ORDER BY s.timestamp DESC";
}

// Pagination
$sqlQuery .= " LIMIT :limit OFFSET :offset";

$stmt = $this->_dbHandle->prepare($sqlQuery);

foreach ($bindParam as $key => $value) {
    $stmt->bindValue($key, $value);
}

$stmt->bindValue(':limit', $limit,
    PDO::PARAM_INT);
$stmt->bindValue(':offset', $offset,
    PDO::PARAM_INT);

$stmt->execute();

// Convert results into objects
$dataSet = [];
while ($row = $stmt->fetch()) {
    $dataSet[] = new SightingsData($row);
}

```

```

    return $dataSet;
}

/**
 * Count sightings with dynamic filters.
 * @param array $params Keys: search, status,
 * species
 * @return int Total matched records
 */
public function countSightingsWithFilters(
    array $params
): int {
    $sqlQuery = "SELECT COUNT(*)
                FROM pets p
                INNER JOIN sightings s
                ON p.id = s.pet_id
                WHERE 1=1";

    $bindParam = [];

    if (!empty($params['search'])) {
        $sqlQuery .= " AND (p.name LIKE :search
                        OR p.species LIKE :search
                        OR p.status LIKE :search
                        OR s.comment LIKE :search
                        OR s.latitude LIKE :search
                        OR s.longitude LIKE :search)";
        $bindParam[':search'] =
            '%' . $params['search'] . '%';
    }

    if (!empty($params['status'])) {
        $sqlQuery .= " AND p.status = :status";
        $bindParam[':status'] = $params['status'];
    }

    if (!empty($params['species'])) {
        $sqlQuery .= " AND p.species = :species";
        $bindParam[':species'] = $params['species'];
    }

    $statement = $this->_dbHandle->prepare($sqlQuery);
    foreach ($bindParam as $key => $value) {
        $statement->bindValue($key, $value);
    }

    $statement->execute();
    return (int)$statement->fetchColumn(0);
}
}

```

9. CreateSightings.php

```

<?php
// Class Model required to inheritance database connections
// and posting location data to the corresponding table
require_once('Database.php');
require_once('LocationDataSets.php');

```

```

/**
 * CreateSightings class designed to create, update and delete sightings
 */
class CreateSightings {
/**
 * @var PDO
 */
private $_dbHandle;

/**
 * Constructor to inheritance the database connections
 */
public function __construct() {
    $this->_dbHandle = Database::getInstance()->getDbConnection();
}

/**
 * @return array return and array of lost pets
 * @throws Exception
 * Get all pets that are currently lost to use for dropdown list
 */
public function getLostPets(): array {
    try {
        // Only get lost pets
        $sql = "SELECT id, name, species, breed, photo_url
                FROM pets
                WHERE status = 'lost'
                ORDER BY name ASC";
        $stmt = $this->_dbHandle->query($sql);
        // Return all lost pets
        return $stmt->fetchAll(PDO::FETCH_ASSOC);
    } catch (PDOException $e) {
        throw new Exception(
            'Database error fetching lost pets: ' . $e->getMessage()
        );
    }
}

/**
 * Method to record a new sighting in the database table sightings.
 * Also updates the locations table by connecting with LocationDataSet.
 * @throws Exception
 */
public function recordSighting(
    $petId, $userId, $comment, $latitude, $longitude
): bool {
    try {
        // Insert the sightings data to the database
        $sql = "INSERT INTO sightings
                (pet_id, user_id, comment, latitude, longitude)
                VALUES
                (:pet_id, :user_id, :comment, :latitude, :longitude)";
        $stmt = $this->_dbHandle->prepare($sql);
        $success = $stmt->execute([
            ':pet_id' => $petId,
            ':user_id' => $userId,
            ':comment' => $comment,
            ':latitude' => $latitude,
            ':longitude' => $longitude
        ]);
    }
}

```

```

// Also insert new location record when a sighting is created
if ($success) {
    $locationDataSet = new LocationDataSet();
    $timestamp = date('Y-m-d H:i:s');
    $locationDataSet->insertLocation(
        $petId, $latitude, $longitude, $timestamp
    );
}

return $success;
} catch (PDOException $e) {
    throw new Exception("Error saving sighting: " . $e->getMessage());
}
}

/**
 * @throws Exception
 * Get all sightings created by a specific user (with pet image).
 */
public function getSightingsByUser($userId): array {
    try {
        // Join the table sightings and pets to get information
        $sql = "SELECT
            s.id,
            p.name AS pet_name,
            p.species,
            p.photo_url,
            s.comment,
            p.photo_url AS photo_url,
            s.latitude,
            s.longitude,
            s.timestamp
        FROM sightings s
        JOIN pets p ON s.pet_id = p.id
        WHERE s.user_id = :user_id
        ORDER BY s.timestamp DESC";

        $stmt = $this->_dbHandle->prepare($sql);
        $stmt->bindValue(':user_id', $userId, PDO::PARAM_INT);
        $stmt->execute();
        return $stmt->fetchAll(PDO::FETCH_ASSOC);
    } catch (PDOException $e) {
        throw new Exception(
            "Error fetching user sightings: " . $e->getMessage()
        );
    }
}

/**
 * @throws Exception
 * Allow owner and user to update sightings and corresponding location.
 */
public function updateSighting($sightingId, $userId, $data): bool {
    try {
        $allowed = ['comment', 'latitude', 'longitude'];
        $fields = [];
        $params = [
            ':id' => $sightingId,
            ':user_id' => $userId
        ];
    }

    // Prepare valid fields and parameters for SQL update

```

```

foreach ($data as $key => $value) {
    if (in_array($key, $allowed, true)) {
        $fields[] = "$key = :$key";
        $params["$key"] = $value;
    }
}

// Nothing to update
if (empty($fields)) return false;

// Update sightings table with new info and timestamp
$sql = "UPDATE sightings
        SET " . implode(', ', $fields) . ",
            timestamp = datetime('now')
        WHERE id = :id AND user_id = :user_id";
$stmt = $this->_dbHandle->prepare($sql);
$stmt->execute($params);

if ($stmt->rowCount() === 0) return false;

// Update or insert location to locations table
$petStmt = $this->_dbHandle->prepare(
    "SELECT pet_id FROM sightings WHERE id = :id"
);
$petStmt->execute([':id' => $sightingId]);
$petId = $petStmt->fetchColumn();

if ($petId) {
    $timestamp = date('Y-m-d H:i:s');
    $latitude = $data['latitude'] ?? null;
    $longitude = $data['longitude'] ?? null;

    // Insert new location record
    $locSql = "INSERT INTO locations
                (pet_id, latitude, longitude, timestamp)
                VALUES
                (:pet_id, :latitude, :longitude, :timestamp)";
    $locStmt = $this->_dbHandle->prepare($locSql);
    $locStmt->execute([
        ':pet_id' => $petId,
        ':latitude' => $latitude,
        ':longitude' => $longitude,
        ':timestamp' => $timestamp
    ]);
}

return true;
} catch (PDOException $e) {
    throw new Exception("Error updating sighting: " . $e->getMessage());
}
}

/**
 * @throws Exception
 * Allow owner and user to delete their sightings by matching user id.
 */
public function deleteSighting($sightingId, $userId): bool {
    try {
        // Get pet_id linked to this sighting
        $petStmt = $this->_dbHandle->prepare(
            "SELECT pet_id FROM sightings

```

```

        WHERE id = :id AND user_id = :user_id"
    );
    $petStmt->execute([
        ':id' => $sightingId,
        ':user_id' => $userId
    ]);
    $petId = $petStmt->fetchColumn();

    if (!$petId) return false;

    // Delete the sighting
    $sql = "DELETE FROM sightings
        WHERE id = :id AND user_id = :user_id";
    $stmt = $this->_dbHandle->prepare($sql);
    $stmt->bindValue(':id', $sightingId, PDO::PARAM_INT);
    $stmt->bindValue(':user_id', $userId, PDO::PARAM_INT);
    $stmt->execute();

    // If deleted, remove corresponding location
    if ($stmt->rowCount() > 0) {
        $delLoc = $this->_dbHandle->prepare(
            "DELETE FROM locations WHERE pet_id = :pet_id"
        );
        $delLoc->execute([':pet_id' => $petId]);
        return true;
    }

    return false;
} catch (PDOException $e) {
    throw new Exception("Error deleting sighting: " . $e->getMessage());
}
}
}

```

10. LocationData.php

```

<?php

/**
 * Class LocationData
 *
 * Works as a data model for LocationDataSet class
 */
class LocationData {
    /**
     * @var int $_id unique location id generated in db
     * @var string $_pet_id foreign key to get pet_id and sightings locations
     * @var string $_latitude sightings latitude
     * @var string $_longitude sightings longitude
     * @var string $_timestamp sighted time
     */
    protected $_id, $_pet_id, $_latitude, $_longitude, $_timestamp;

    /**
     * LocationData constructor.
     * Initializes the object with data from a database row.
     */
    public function __construct($dbRow) {
        $this->_id = $dbRow['id'];
    }
}

```

```

        $this->_pet_id = $dbRow['pet_id'];
        $this->_latitude = $dbRow['latitude'];
        $this->_longitude = $dbRow['longitude'];
        $this->_timestamp = $dbRow['timestamp'];
    }

    // Get the location id
    public function getLocationId() {
        return $this->_id;
    }

    // Get the pet ID
    public function getPetId() {
        return $this->_pet_id;
    }

    // Get the location latitude
    public function getLatitude() {
        return $this->_latitude;
    }

    // Get the location longitude
    public function getLongitude() {
        return $this->_longitude;
    }

    // Get the sighted time when location was recorded
    public function getTimestamp() {
        return $this->_timestamp;
    }
}

```

11. LocationDataSets.php

```

<?php
// Inheritance methods and database connections from below class models
require_once('Database.php');
require_once('LocationData.php');

/**
 * Class LocationDataSet
 * Handles syncing and inserting pet location data
 * from the sightings table into the location table.
 */
class LocationDataSet {
    /**
     * @var PDO
     */
    private $_dbHandle;

    /**
     * Constructor to inheritance database connection
     */
    public function __construct() {
        $this->_dbHandle = Database::getInstance()->getdbConnection();
    }

    /**
     * Insert a single new location record (used after new sighting is

```

```

created).
    * Allows updating the location table when user creates new sightings.
    * @throws Exception
    */
    public function insertLocation(
        $petId, $latitude, $longitude, $timestamp = null
    ): bool {
        try {
            // Insert location data and upload current date/time when sighted
            // COALESCE defaults to current time if timestamp not given
            $sql = "INSERT INTO locations
                    (pet_id, latitude, longitude, timestamp)
                    VALUES
                    (:pet_id, :latitude, :longitude, :timestamp)";
            $stmt = $this->_dbHandle->prepare($sql);
            return $stmt->execute([
                ':pet_id' => $petId,
                ':latitude' => $latitude,
                ':longitude' => $longitude,
                ':timestamp' => $timestamp
            ]);
        } catch (PDOException $e) {
            throw new Exception(
                "Error inserting location: " . $e->getMessage()
            );
        }
    }
}

```

D) Controller and View files

userAuthentication.php

```
<?php
// Inheritance the UserAuthentication model class to
// implement authentication
require_once('Models/UserAuthentication.php');

/**
 * We will use UserAuthentication class function
 * hashUserPassword to hash passwords in the database.
 *
 * Note: For ease and testing purposes we will
 * 1. Use same password for all Owners:
 *    07m!eX#2vLp9@zFq
 * 2. Use same password for all Users:
 *    U$4PzN!w8Hq@1rBx
 */
$userAuthentication = new UserAuthentication();
$rows = $userAuthentication->hasUserPassword(
    '07m!eX#2vLp9@zFq', 'U$4PzN!w8Hq@1rBx'
);

/**
 * Test if the hashing process is successful and print a
 * message (for developer testing) showing how many rows
 * have been hashed correctly.
 *
 * In this project, the expected number of rows is 210.
 */
echo "Updated password hashes for $rows users.";
```

index.php

```
<?php
session_start();
$view = new stdClass();
$view->title = "Home Page";

// Show the home page view
require_once('Views/index.phtml');
```

index.phtml associated with index.php

```
<?php require('template/header.phtml') ?>

<!-- Demo Hero Section -->
<section class="hero-section mt-5 pt-5">
    <div class="container">
        <h1 class="hero-title">Welcome to Pet Watch</h1>
        <p class="hero-subtitle">
            Let's help each other find lost pets whom we love very much!<br>
            Join our community of caring pet owners today.
        </p>
    <div class="hero-buttons">
```

```

    <a href="/viewSightings.php" class="hero-btn hero-btn-primary">
      View Sightings
    </a>
    <a href="/createSightings.php" class="hero-btn hero-btn-secondary">
      Report a Sighting
    </a>
  </div>
</div>
</section>

<!-- Demo stats section -->
<section class="stats-section">
  <div class="container">
    <div class="row">
      <div class="col-md-4 stat-item">
        <div class="stat-number">1000+</div>
        <div class="stat-label">Pets Reunited</div>
      </div>
      <div class="col-md-4 stat-item">
        <div class="stat-number">5000+</div>
        <div class="stat-label">Active Users</div>
      </div>
      <div class="col-md-4 stat-item">
        <div class="stat-number">24/7</div>
        <div class="stat-label">Community Support</div>
      </div>
    </div>
  </div>
</section>

<!-- Demo Features Section -->
<section class="features-section">
  <div class="container">
    <h2 class="section-title">How Can We Help?</h2>

    <div class="row">

      <!-- Dog Card -->
      <div class="col-xs-12 col-lg-4 mb-5">
        <div class="card feature-card">
          
          <div class="card-body feature-card-body">
            <h5 class="feature-card-title">Missing Your Dog?</h5>
            <p class="feature-card-text">
              Pet Watch is a rapidly growing website that users trust.
              Every day, people share their success stories of finding their dogs
              through our platform. Don't wait - create a sighting now to help
              reunite with your lost dog.
            </p>
            <a href="/viewSightings.php" class="feature-btn">View Sightings</a>
          </div>
        </div>
      </div>

      <!-- Cat Card -->
      <div class="col-xs-12 col-lg-4 mb-5">
        <div class="card feature-card">
          
          <div class="card-body feature-card-body">
            <h5 class="feature-card-title">Report A Missing Cat</h5>
            <p class="feature-card-text">

```

```

        Your cat might be nearby. Report a sighting now and help reunite
        with
        your furry companion. Pet Watch connects a community of cat lovers
        who
        share updates, sightings, and tips to ensure lost cats are found
        quickly
        and safely.
    </p>
    <a href="/createSightings.php" class="feature-btn">Create
    Sightings</a>
</div>
</div>
</div>

<!-- Bird Card -->
<div class="col-xs-12 col-lg-4 mb-5">
    <div class="card feature-card">
        
        <div class="card-body feature-card-body">
            <h5 class="feature-card-title">Bird Flew Away?</h5>
            <p class="feature-card-text">
                Missing your beloved bird? Even small sightings can make a big
                difference. Share updates and sightings on Pet Watch to help bring
                your feathered companion back home. Login now to report missing
                bird!
            </p>
            <a href="/viewSightings.php" class="feature-btn">Search Missing
            Pets</a>
        </div>
    </div>
</div>
</div>
</div>
</section>

<!--Load the footer view -->
<?php require('Views/template/footer.phtml') ?>

```

login.php

```

<?php
// Session setup for an hour
$lifetime = 3600;

/**
 * Cookie last an hour
 * Cookie is available to the whole website
 * Sends the cookie only over HTTPS if the connection is secure
 * httponly prevents access to JavaScript to prevent XSS attack
 * samesite: lax prevents CSRF (Cross Site Request Forgery)
 */
session_set_cookie_params([
    'lifetime' => $lifetime,
    'path' => '/',
    'secure' => isset($_SERVER['HTTPS']),
    'httponly' => true,
    'samesite' => 'Lax'
]);

```

```

session_start();

$view = new stdClass();
$view->title = "Login";
$view->loginError = '';
$view->successMessage = '';

// Include the UserAuthentication class to check users login data
// against the database
require_once('Models/UserAuthentication.php');

$userAuthentication = new UserAuthentication();

// When logout button is clicked, destroy the cookie session
if (isset($_GET['action']) && $_GET['action'] === 'logout') {
    $_SESSION = [];
    session_destroy();
    $view->successMessage = 'You have been logged out successfully.';
}

/**
 * Implement anti-spam feature
 * Allow up to 3 failed login attempts, if three attempt failed,
 * block user for 2 min even when they type right password, it won't
 * allow to login to prevent spam
 */
if (!isset($_SESSION['login_attempts'])) {
    $_SESSION['login_attempts'] = 0;
    $_SESSION['last_attempt_time'] = time();
}

// If user exceeded 3 failed attempts
if ($_SESSION['login_attempts'] >= 3) {
    $elapsed = time() - $_SESSION['last_attempt_time'];

    // If less than 2 minutes (120 seconds) have passed
    if ($elapsed < 120) {
        // Show how many second they have to wait before trying again
        $remaining = 120 - $elapsed;
        $view->loginError = "Too many login attempts. Please wait
        {$remaining} seconds before trying again.";
        require_once('Views/login.phtml');
        exit();
    } else {
        // Reset counter after cooldown period, now if user type right
        // password, they will be availed to log in
        $_SESSION['login_attempts'] = 0;
    }
}

// If the cookie is not set and user is not logged-in
if (isset($_POST['login'])) {
    // Get the username and password user posted
    $username = trim($_POST['username'] ?? '');
    $password = trim($_POST['password'] ?? '');

    // sanitize and validate inputs
    $username = htmlspecialchars($username, ENT_QUOTES);
    $password = htmlspecialchars($password, ENT_QUOTES);

    // Verify against the database information

```

```

$user = $userAuthentication->verifyLogin($username, $password);

// If login credential match, get the user data
if ($user) {
    $_SESSION['loggedin'] = true;
    $_SESSION['username'] = $user->getUsername();
    $_SESSION['user_id'] = $user->getUserId();
    $_SESSION['role'] = $user->getRole();

    // Reset failed attempts after successful login
    $_SESSION['login_attempts'] = 0;

    $view->successMessage = 'Login successful!';
} else {
    // Increment failed attempt count and record the time
    $_SESSION['login_attempts']++;
    $_SESSION['last_attempt_time'] = time();
    $view->loginError = 'Invalid username or password.';
}
}

// Show them the login.phtml where login form and logout form view
// is hold
require_once('Views/login.phtml');

```

login.phtml associated with login.php

```

<?php
// Include the header view
require('Views/template/header.phtml');
?>

<!--Check if a user is logged in-->
<?php if (isset($_SESSION['loggedin']) && $_SESSION['loggedin'] === true):
?>

<!-- Logged In: Show Welcome Message with logout button -->
<div class="container mt-5" style="min-height: 70vh">
    <div class="row">
        <div class="col-md-8 offset-md-2">
            <div class="welcome-card">

                <!--Show the user login succes message-->
                <?php if (!empty($view->successMessage)): ?>
                    <div class="alert alert-success mb-4">
                        <?= htmlspecialchars($view->successMessage) ?>
                    </div>
                <?php endif; ?>

                <!--Logged in success, welcome the user by their name -->
                <h2 class="welcome-title">
                    Welcome back, <?= htmlspecialchars($_SESSION['username']) ?>!
                </h2>
                <p class="welcome-subtitle">
                    You are successfully logged in to Pet Watch.
                </p>

                <!--Show some page buttons links so user can navigate-->
                <div class="quick-links">

```

```

        <a href="/index.php" class="quick-link-btn">Go to Home</a>
        <a href="/viewSightings.php" class="quick-link-btn">View
            Sightings</a>
        <a href="/mypet.php" class="quick-link-btn">My Pets</a>
    </div>

    <hr class="my-4">
    <a href="/login.php?action=logout" class="btn btn-danger logout-btn">
        Logout
    </a>

    </div>
</div>
</div>
</div>

<!-- Not Logged In: Show Login Form -->
<?php else: ?>

<div class="container login-form">
    <div class="row">
        <div class="col-lg-6 mx-auto">
            <div class="login-card">
                <h4 class="login-title">Sign In</h4>

                <!-- Show general error message -->
                <?php if (!empty($view->errorMessage)): ?>
                    <div class="alert alert-warning text-center shadow-sm mb-4">
                        <?= htmlspecialchars($view->errorMessage) ?>
                    </div>
                <?php endif; ?>

                <!--Show the success message if login is successful-->
                <?php if (!empty($view->successMessage)): ?>
                    <div class="alert alert-success text-center">
                        <?= htmlspecialchars($view->successMessage) ?>
                    </div>
                <?php endif; ?>

                <!--Show relevant error if not successful-->
                <?php if (!empty($view->loginError)): ?>
                    <div class="alert alert-danger text-center">
                        <?= htmlspecialchars($view->loginError) ?>
                    </div>
                <?php endif; ?>

                <!--If not logged in, show login form-->
                <form method="post" action="/login.php">

                    <!--Post the username-->
                    <div class="mb-3">
                        <label for="username" class="form-label">Username</label>
                        <input type="text" name="username" id="username"
                            class="form-control" placeholder="Enter your username"
                            value="<?= htmlspecialchars($_POST['username']) ?? '' ?>"
                            required>
                    </div>

                    <!--Post the password-->
                    <div class="mb-3">
                        <label for="password" class="form-label">Password</label>

```

```
<input type="password" name="password" id="password"
      class="form-control" placeholder="Enter your password"
      required>
</div>

<!--This doesn't check captcha, demo only-->
<div class="form-check mb-3">
  <input type="checkbox" class="form-check-input"
        name="captcha" value="Bike" id="humanCheck">
  <label for="humanCheck" class="form-check-label">
    I'm not a robot
  </label>
</div>

<!-- Submit -->
<div class="d-grid">
  <button type="submit" name="login" class="btn btn-success btn-lg">
    Login
  </button>
</div>
</form>

<!--Register placeholder-->
<div class="text-center mt-3">
  <small class="text-muted">
    Don't have an account? Contact support.
  </small>
</div>
</div>
</div>
</div>
</div>
</div>

<?php endif; ?>

<!--Include the footer view -->
<?php require('Views/template/footer.phtml'); ?>
```

viewSightings.php

```
<?php
// Start the cookie session for logged-in user
session_start();

// Inheritance model classes to implement different functions and db
connections
require_once('Models/Database.php');
require_once('Models/LocationDataSets.php');
require_once('Models/ViewSightingsDataSets.php');

$view = new stdClass();

/**
 * Highlights search terms in a given text string.
 * If input matches any results, highlight it in yellow and bold it (custom
CSS)
 */
function highlightSearchTerm($text, $searchTerm) {
    // If there's no search term or text, just return it as is
```

```

if (empty($searchTerm) || empty($text)) {
    return htmlspecialchars($text);
}

// Escape special regex characters so they don't break the regex pattern
$searchTerm = preg_quote($searchTerm, '/');

// Replace matched text with a highlighted version and add custom css
$highlighted = preg_replace_callback(
    '/(' . $searchTerm . ')/i',
    function ($match) {
        // Wrap matching word in <mark> for highlighting
        return '<mark class="search-highlight">' .
            htmlspecialchars($match[0]) . '</mark>';
    },
    $text
);

// Return highlighted version or safe user input text if nothing matches
return $highlighted ?? htmlspecialchars($text);
}

// Set how many results to show per page
$limit = 10;

// Get the current page number from the URL (default to 1)
$page = filter_input(
    INPUT_GET, 'page', FILTER_VALIDATE_INT,
    ['options' => ['default' => 1, 'min_range' => 1]]
);
// Calculate how many records to skip (used for pagination)
$offset = ($page - 1) * $limit;

// Get the search term from the URL if available
$searchTerm = trim($_GET['search'] ?? '');
$view->searchTerm = htmlspecialchars($searchTerm);

// Get filter and sort values from the URL (species, status, order, date)
$filterStatus = trim($_GET['filter_status'] ?? '');
$filterSpecies = trim($_GET['filter_species'] ?? '');
$sortOrder = trim($_GET['sort_order'] ?? '');
$sortDate = trim($_GET['sort_date'] ?? '');

// Save filters so form inputs can stay filled after submission
$view->filters = [
    'status' => $filterStatus,
    'species' => $filterSpecies
];

// Figure out which sorting is used
if (!empty($sortDate)) {
    $view->sort = $sortDate;
} elseif (!empty($sortOrder)) {
    $view->sort = $sortOrder;
} else {
    $view->sort = '';
}

// Create model object to handle database operation
$sightingsDataSet = new ViewSightingsDataSet();

```

```
// Combine all filter and sort settings into one array
$params = [
    'search' => $searchTerm,
    'status' => $filterStatus,
    'species' => $filterSpecies,
    'sort_order' => $sortOrder,
    'sort_date' => $sortDate
];

// Get total records for pagination that matched filters
$totalRecords = $sightingsDataSet->countSightingsWithFilters($params);
$totalPages = ceil($totalRecords / $limit);

// Save pagination data for the view
$view->page = $page;
$view->totalPages = $totalPages;
$view->limit = $limit;

// Fetch paginated sightings data
$view->sightingsDataSet = $sightingsDataSet->fetchSightingsWithFilters(
    $params, $limit, $offset
);

// Create a message to show above results
// Example: 152 total result(s). Showing page 1 of 16.
if ($totalRecords == 0) {
    $view->dbMessage = !empty($searchTerm)
        ? "No results found matching '" . htmlspecialchars($searchTerm) . "'"
        : "No results found.";
} else {
    $view->dbMessage = "$totalRecords total result(s)";
    if (!empty($searchTerm)) {
        $view->dbMessage .= " found matching '" .
            htmlspecialchars($searchTerm) . "'";
    }
    if ($totalPages > 1) {
        $view->dbMessage .= ". Showing page $page of $totalPages.";
    }
}

// Include the view
require_once('Views/viewSightings.phtml');
```

viewSightings.phtml associated with viewSightings.php

```
<?php
require('template/header.phtml');
?>

<div class="container mt-5 mb-3">
    <!-- Search and Filters -->
    <form class="row g-2 align-items-center" role="search"
        action="/viewSightings.php" method="get">

        <div class="col-md-6 col-sm-12">
            <div class="input-group">
                <input type="search" class="form-control"
                    placeholder="Search by pet name, species, comment, lat, long
etc."
                >
```

```

        name="search"
        value="<?=  
        htmlspecialchars($view->searchTerm ?? ' ') ?>"
        aria-label="Search">
        <button class="btn btn-success" type="submit">Search</button>
    </div>
</div>

<div class="col-md-6 col-sm-12">
    <div class="row g-2">
        <!-- Sort by name -->
        <div class="col-md-3 col-6">
            <select name="sort_order" class="form-select">
                <option value="">Sort by Name</option>
                <option value="name_asc"
                    <?=  
                        ($view->sort === 'name_asc') ? 'selected' : ' ' ?>>
                    A-Z
                </option>
                <option value="name_desc"
                    <?=  
                        ($view->sort === 'name_desc') ? 'selected' : ' ' ?>>
                    Z-A
                </option>
            </select>
        </div>

        <!-- Filter by lost/found -->
        <div class="col-md-3 col-6">
            <select name="filter_status" class="form-select">
                <option value="">Pet Status</option>
                <option value="lost"
                    <?=  
                        ($view->filters['status'] === 'lost') ? 'selected' : ' '
                        ?>>
                    Lost
                </option>
                <option value="found"
                    <?=  
                        ($view->filters['status'] === 'found') ? 'selected' : ' '
                        ?>>
                    Found
                </option>
            </select>
        </div>

        <!-- Filter by upload date -->
        <div class="col-md-3 col-6">
            <select name="sort_date" class="form-select">
                <option value="">Upload Date</option>
                <option value="latest"
                    <?=  
                        ($view->sort === 'latest') ? 'selected' : ' ' ?>>
                    Latest
                </option>
                <option value="oldest"
                    <?=  
                        ($view->sort === 'oldest') ? 'selected' : ' ' ?>>
                    Oldest
                </option>
            </select>
        </div>

        <!-- Filter by species -->
        <div class="col-md-3 col-6">
            <select name="filter_species" class="form-select">
                <option value="">All Types</option>
                <option value="Dog">

```

```

        <?=( $view->filters['species'] === 'Dog') ? 'selected' : ''
        ?>>
        Dog
    </option>
    <option value="Cat"
        <?=( $view->filters['species'] === 'Cat') ? 'selected' : ''
        ?>>
        Cat
    </option>
    <option value="Bird"
        <?=( $view->filters['species'] === 'Bird') ? 'selected' : ''
        ?>>
        Bird
    </option>
</select>
</div>
</div>
</div>

<div class="col-12 text-end mt-2">
    <button class="btn btn-outline-success" type="submit">
        Apply Filters
    </button>
</div>
</form>

<!-- Feedback -->
<div class="mt-2">
    <p class="text-muted mb-0">
        <?=( htmlspecialchars($view->dbMessage ?? '') ) ?>
    </p>
</div>
</div>

<!-- Sightings list -->
<div class="container">
    <?php if (empty($view->sightingsDataSet)): ?>
        <div class="alert alert-info mt-4" role="alert">
            <strong>No sightings found.</strong>
            Try adjusting your search filters.
        </div>
    <?php else: ?>
        <?php foreach ($view->sightingsDataSet as $sightingsData): ?>
            <?php
                $searchTerm = $view->searchTerm ?? '';
                $status = strtolower($sightingsData->getPetStatus());
            ?>
            <div class="sighting-card">
                <div class="sighting-card-header">
                    <span style="font-size: 18px;">
                        <?=( highlightSearchTerm($sightingsData->getPetName(), $searchTerm) ) ?>
                    </span>
                    <span class="status-badge <?=( $status ) ?>">
                        <?=( highlightSearchTerm($sightingsData->getPetStatus(), $searchTerm) ) ?>
                    </span>
                </div>
                <div class="sighting-card-body">
                    <div class="row">
                        <div class="col-md-3 col-sm-12 text-center mb-3 mb-md-0">

```

```

        
    </div>

    <div class="col-md-9 col-sm-12">
        <div class="sighting-info">

            <div class="info-row">
                <span class="info-label">Species:</span>
                <span class="info-value">
                    <?= highlightSearchTerm($sightingsData->getPetSpecies(),
                        $searchTerm) ?>
                </span>
            </div>

            <div class="info-row">
                <span class="info-label">Comment:</span>
                <div class="info-value comment-box">
                    <?= highlightSearchTerm($sightingsData->getSightingsComment(),
                        $searchTerm) ?>
                </div>
            </div>

            <div class="info-row">
                <span class="info-label">Location:</span>
                <span class="info-value coordinates">
                    Lat: <?= htmlspecialchars($sightingsData->getSightingsLatitude())
                        ?>,
                    Lng: <?= htmlspecialchars($sightingsData->
                        getSightingsLongitude()) ?>
                </span>
            </div>

            <div class="info-row">
                <span class="info-label">Sighted:</span>
                <span class="info-value timestamp">
                    <?= htmlspecialchars($sightingsData->getSightingsTimestamp()) ?>
                </span>
            </div>

        </div>
    </div>
</div>
</div>
<?php endforeach; ?>
<?php endif; ?>

<!-- Pagination -->
<?php if ($view->totalPages > 1): ?>
    <nav aria-label="Pagination" class="mt-4">
        <ul class="pagination justify-content-center">
            <!-- Query string for pagination and filter links
                This ensures when navigating between different view page,
                current search, filter, and sort options remain applied
            -->
            <?php
                $queryParams = [];

                // Add the search term to the query if provided by user

```

```

if (!empty($view->searchTerm))
    $queryParams[] = 'search=' . urlencode($view->searchTerm);
// Add selected status filter if provided by user
if (!empty($view->filters['status']))
    $queryParams[] = 'filter_status=' . urlencode($view->filters['status']);
// add selected species if provided by user
if (!empty($view->filters['species']))
    $queryParams[] = 'filter_species=' . urlencode($view->filters['species']);
// add the filter oldest or latest to the query
if (!empty($view->sort)) {
    if (in_array($view->sort, ['latest', 'oldest'])) {
        $queryParams[] = 'sort_date=' . urlencode($view->sort);
    } else {
        $queryParams[] = 'sort_order=' . urlencode($view->sort);
    }
}

//combine all query parameters into a single URL string which
// allows to keep all th filter
// & allow us to do this in all pagination pages
$queryString = !empty($queryParams)
    ? implode('&', $queryParams) . '&'
    : '';
// Define the base URL which handle the logic (controller
    viewSightings.php)
$baseUrl = 'viewSightings.php?' . $queryString;
?>

<!-- Previous -->
<li class="page-item <?=( $view->page <= 1 ) ? 'disabled' : '' ?>">
    <!--If current page is more than one, activate previous button-->
    <?php if ( $view->page > 1): ?>
        <a class="page-link"
            href="<?=$baseUrl ?>page=<?=$view->page - 1 ?>">&laquo;
            Previous</a>
    <?php else: ?>
        <span class="page-link">&laquo; Previous</span>
    <?php endif; ?>
</li>

<!-- Page Numbers calculation-->
<?php
// Determine which page to display around the current page
// (up to two before and after)
$start = max(1, $view->page - 2);
$end = min($view->totalPages, $view->page + 2);
//Loop through the range and render the page number
for ($i = $start; $i <= $end; $i++):
    $active = ($i == $view->page) ? 'active' : '';
?>
<li class="page-item <?=$active ?>">
    <!--Display the active page number and make it non-clickable -->
    <?php if ($active): ?>
        <span class="page-link"><?=$i ?></span>
    <!-- Display other page numbers as clickable links -->
    <?php else: ?>
        <a class="page-link" href="<?=$baseUrl ?>page=<?=$i ?>"><?=$i
            ?></a>
    <?php endif; ?>

```

```

        </li>
    <?php endfor; ?>

    <!-- Next button for pagination -->
    <li class="page-item <?=( $view->page >= $view->totalPages) ?
        'disabled' : '' ?>">
        <!-- If the current page is not the last one, enable the next button-->
        <?php if ( $view->page < $view->totalPages): ?>
            <a class="page-link"
                href="<?=$baseUrl ?>page=<?=$view->page + 1 ?>">Next &raquo;</a>
        <?php else: ?>
            <span class="page-link">Next &raquo;</span>
        <?php endif; ?>
    </li>
</ul>
</nav>
<?php endif; ?>
</div>

<?php require('template/footer.phtml'); ?>

```

createSightings.php

```

<?php
session_start();
// Inheritance the model class CreateSightings.php to use its methods
require_once('Models/CreateSightings.php');

$view = new stdClass();
$view->title = "Create Sightings";
$view->successMessage = ''; // Empty variable initiated to show success
$view->errorMessage = ''; // variable to show error
$view->pets = []; // Initialise pets array to hold objects later
$view->sightings = []; // Initialise sightings array to hold object later
$view->selectedPetId = null; // Added to track selected pet

// Check if a user is not logged in
if (empty($_SESSION['user_id'])) {
    $view->errorMessage = "You must log in to create a sighting.";
    require_once('Views/login.phtml');
    exit();
}

// Take the user id of the logged-in user (cast it to an int to make sure
// user_id is always integer number)
$userId = (int)$_SESSION['user_id'];
// Create an object of type CreateSightings class
$createSightingsModel = new CreateSightings();

// now on the view we use pets array to hold the lost pet.
try {
    // The function get lost pet is used to get the lost pets from the
    database.
    $view->pets = $createSightingsModel->getLostPets();
} catch (Exception $e) {

```

```

$view->errorMessage = $e->getMessage();
}

/**
 * Check posted input from the createsightings.phtml and validate
 * against the criteria set below
 * Comment up to 100 characters
 * Latitude is float and -90 to 90
 * Longitude is float and -180 to 180
 */
function validateSightingData($input): array {
    // Trim input and escape HTML for security (prevents XSS)
    $comment = trim($input['comment'] ?? '');
    $lat = $input['latitude'] ?? null;
    $lng = $input['longitude'] ?? null;

    $errors = [];

    // Validate latitude and longitude (must be numeric)
    if (!is_numeric($lat) || !is_numeric($lng)) {
        $errors[] = "Latitude and longitude must be numeric.";
    } else {
        $lat = (float)$lat;
        $lng = (float)$lng;

        if ($lat < -90 || $lat > 90 || $lng < -180 || $lng > 180) {
            $errors[] = "Invalid coordinates. Latitude must be between -90 and 90, "
                . "longitude between -180 and 180.";
        }
    }

    // Validate comment length only (allow all characters)
    if (mb_strlen($comment) > 200) {
        $errors[] = "Invalid comment. Must be up to 200 characters.";
    }

    // Escape HTML to prevent XSS when displaying later
    $comment = htmlspecialchars($comment, ENT_QUOTES, 'UTF-8');

    return [$comment, $lat, $lng, $errors];
}

// Handle form submission
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    // Check if the user selected any pet from the lost pet list
    // If user selected any pet, post the pet_id to the view to show pet image
    if (
        isset($_POST['pet_id']) &&
        // When submit, update, and delete is not set.
        !isset($_POST['submit_sighting']) &&
        !isset($_POST['update_sighting']) &&
        !isset($_POST['delete_sighting'])
    ) {
        // This will allow us to show the selected image on the view
        $view->selectedPetId = (int)$_POST['pet_id'];
    }

    // check if the action is submit sightings, if yes, add sightings to the
    db
    if (isset($_POST['submit_sighting'])) {
        // filter the pet id that has been selected and make sure its always int

```

```

$petId = filter_input(
    INPUT_POST, 'pet_id', FILTER_SANITIZE_NUMBER_INT
);

// Use the validateSightingsData function to check if the user has given
// valid data
[$comment, $lat, $lng, $errors] = validateSightingData($_POST);

// If there is any error such as invalid lat, long, post it the view
if ($errors) {
    $view->errorMessage = implode(' ', $errors);
}
// No errors found, attempt to submit the sighting
else {
    try {
        // Add the sightings to the database and show it to the view
        $successOrFail = $createSightingsModel->recordSighting(
            $petId, $userId, $comment, $lat, $lng
        );
        $view->successMessage = $successOrFail
            ? "Sighting added."
            : "Failed to add sightings";
    } catch (Exception $e) {
        $view->errorMessage = $e->getMessage();
    }
}
}

// check if the action is update_sighting if it's not submit-sightings
elseif (isset($_POST['update_sighting'])) {
    // cast the pet id from string to int, and update it to the sighting
    $id = (int)$_POST['update_sighting'];

    // Now we check whether the updated sightings pet details meet
    // the sightings data requirement
    [$comment, $lat, $lng, $errors] = validateSightingData(
        $_POST['sightings'][$id] ?? []
    );

    // If there is any error such as invalid lat, long, post it the view
    if ($errors) {
        $view->errorMessage = implode(' ', $errors);
    }
    // No error found, attempt to update the pet
    else {
        try {
            // Prepare the data array for the model
            $data = [
                'comment' => $comment,
                'latitude' => $lat,
                'longitude' => $lng
            ];

            // Update the sightings data in the database
            $successOrFail = $createSightingsModel->updateSighting(
                $id, $userId, $data
            );
            $view->successMessage = $successOrFail
                ? "Sighting updated."
                : "No changes made.";
        } catch (Exception $e) {

```

```

        $view->errorMessage = $e->getMessage();
    }
}

// Delete a sighting from the database
elseif (isset($_POST['delete_sighting'])) {
    $sightingId = (int)$_POST['delete_sighting'];

    // Use the deleteSighting function from the model class
    // and delete the requested sighting id
    try {
        $delete = $createSightingsModel->deleteSighting(
            $sightingId, $userId
        );
        $view->successMessage = $delete
            ? "Sighting deleted successfully."
            : "Failed to delete sighting.";
    } catch (Exception $e) {
        $view->errorMessage = $e->getMessage();
    }
}

// Fetch user's own sightings on the view when logged in and check error
try {
    // Get the sightings of a logged-in user
    $view->sightings = $createSightingsModel->getSightingsByUser(
        $userId
    );
} catch (Exception $e) {
    $view->errorMessage = $e->getMessage();
}

// Load the main view
require_once('Views/createSightings.phtml');

```

createSightings.phtml associated with createSightings.php

```

<?php require_once('Views/template/header.phtml'); ?>

<!--If creating a sightings is successful show succesMessage,
otherwise show error message-->
<div class="row justify-content-center mt-5">
    <div class="col-12 col-md-6">
        <?php if (!empty($view->successMessage)): ?>
            <div class="alert alert-success text-center">
                <?= htmlspecialchars($view->successMessage) ?>
            </div>
        <?php elseif (!empty($view->errorMessage)): ?>
            <div class="alert alert-danger text-center">
                <?= htmlspecialchars($view->errorMessage) ?>
            </div>
        <?php endif; ?>
    </div>
</div>

<!--Report a new sightings container-->
<div class="container my-5">

```

```

<div class="row justify-content-center pt-5">
  <div class="col-12 col-md-6 shadow-lg p-4 mb-5 bg-white rounded">
    <h2 class="mb-4">Report a New Sighting</h2>

    <!-- Determine which pet id the user selected.
         Using this, we will display the image of the chosen pet when
selected.
         This demonstrates how to efficiently retrieve and display data from
DB. -->
    <?php
    $selectedPet = null;
    $selectedPetId = $view->selectedPetId ?? null;

    // If user selected a pet, show that pet
    if (!empty($selectedPetId) && !empty($view->pets)) {
      foreach ($view->pets as $pet) {
        if ((int)$pet['id'] === (int)$selectedPetId) {
          $selectedPet = $pet;
          break;
        }
      }
    }
    ?>

    <!-- Create sightings form -->
    <form action="/createSightings.php" method="post">

      <div class="mb-3">
        <label for="pet_id" class="form-label">Missing Pets</label>
        <!--Each time user selects a pet, submit to update displayed image-->
        <select name="pet_id" id="pet_id" class="form-select"
              required onchange="this.form.submit()">
          <option value="" disabled <?=$selectedPetId ? '' : 'selected' ?>>
            --Choose a pet--
          </option>
          <!--Loop through all pets-->
          <?php foreach ($view->pets as $pet): ?>
            <option value="<?= htmlspecialchars($pet['id']) ?>"
                  <?= ($selectedPetId == $pet['id']) ? 'selected' : '' ?>>
              <?= htmlspecialchars($pet['name']) ?>
              (<?= htmlspecialchars($pet['species']) ?>)
            </option>
          <?php endforeach; ?>
        </select>
      </div>

      <!--Show selected pet image-->
      <?php if ($selectedPet && !empty($selectedPet['photo_url'])): ?>
        <div class="mb-3 text-left">
          
        </div>
      <?php endif; ?>

      <!--Post sightings comment-->
      <div class="mb-3">
        <label for="comment" class="form-label">Comment</label>
        <textarea name="comment" id="comment" class="form-control text-left"
              rows="2" placeholder="Briefly describe the pet conditions">
          <?= htmlspecialchars($ POST['comment'] ?? '') ?>
        </div>
      </div>
    </form>
  </div>
</div>

```

```

</textarea>
</div>

<div class="mb-3">
  <label class="form-label fw-bold">Enter lat/lng where you saw the
    pet.</label>
  <div class="row">

    <!--Latitude-->
    <div class="col-6">
      <label for="latitude" class="form-label">Latitude</label>
      <input type="number" step="any" name="latitude" id="latitude"
        class="form-control" placeholder="Latitude"
        value="<?= htmlspecialchars($_POST['latitude']) ?>"
        required>
    </div>

    <!--Longitude-->
    <div class="col-6">
      <label for="longitude" class="form-label">Longitude</label>
      <input type="number" step="any" name="longitude" id="longitude"
        class="form-control" placeholder="Longitude"
        value="<?= htmlspecialchars($_POST['longitude']) ?>"
        required>
    </div>
  </div>
</div>

<!--Submit-->
<div class="d-grid">
  <button type="submit" name="submit_sighting" class="btn btn-primary">
    Submit Sighting
  </button>
</div>
</form>
</div>
</div>

<!-- Update and delete sightings when logged in -->
<div class="container pb-5 mb-5 mt-5">
  <div class="row">
    <div class="col-lg-10 mx-auto">
      <div class="card shadow-sm border-0 bg-light rounded-3">
        <div class="card-body p-4">
          <h2 class="text-center mb-5 display-6 text-primary fw-semibold">
            Manage My Sightings
          </h2>

          <?php if (!empty($view->sightings)): ?>
          <form action="/createSightings.php" method="post">
            <div class="row mx-2 gy-4">
              <?php foreach ($view->sightings as $sighting): ?>
              <?php $sightingId = $sighting['id']; ?>
              <div class="col-12">
                <div class="card shadow-sm border-0 bg-white rounded-3">
                  <div class="card-body px-3 py-4">

                    <!--Pet name and species-->
                    <h4 class="card-title mb-3 border-bottom pb-2 fw-semibold
                      text-primary">
                      <?= htmlspecialchars($sighting['pet name']) ?>

```

```

        <small class="text-muted">
            (<?= htmlspecialchars($sighting['species']) ?>)
        </small>
    </h4>

    <!--Pet image-->
    <?php if (!empty($sighting['photo_url'])): ?>
        <div class="mb-3 text-left">
            
        </div>
    <?php endif; ?>

    <div class="row g-3">

        <!--Comment-->
        <div class="col-md-12">
            <label class="form-label small text-muted">Comment</label>
            <textarea name="sightings[<?= $sightingId ?>][comment]"
                class="form-control shadow-sm" rows="2">
                <?= htmlspecialchars($sighting['comment'] ?? '') ?>
            </textarea>
        </div>

        <!--Latitude-->
        <div class="col-md-6 col-lg-3">
            <label class="form-label small text-muted">Latitude</label>
            <input type="text"
                name="sightings[<?= $sightingId ?>][latitude]"
                value="<?= htmlspecialchars($sighting['latitude'])
                    ?>"
                class="form-control shadow-sm">
        </div>

        <!--Longitude-->
        <div class="col-md-6 col-lg-3">
            <label class="form-label small text-muted">Longitude</label>
            <input type="text"
                name="sightings[<?= $sightingId ?>][longitude]"
                value="<?= htmlspecialchars($sighting['longitude'])
                    ?>"
                class="form-control shadow-sm">
        </div>

        <!--Timestamp-->
        <div class="col-md-6 col-lg-4">
            <label class="form-label small text-muted">Recorded
                at</label>
            <input type="text" class="form-control shadow-sm"
                value="<?= htmlspecialchars($sighting['timestamp'])
                    ?>"
                readonly>
        </div>
    </div>

    <div class="row g-3 mt-4">

        <!--Update-->
        <div class="col-md-6 col-lg-4">
            <button type="submit" name="update_sighting"
                value="<?= $sightingId ?>"

```

```

        class="btn btn-success w-100 fw-bold shadow-sm">
            <i class="bi bi-pencil-square"></i> Update
        </button>
    </div>

    <!--Delete-->
    <div class="col-md-6 col-lg-4">
        <button type="submit" name="delete_sighting"
            value="<?= $sightingId ?>"
            class="btn btn-outline-danger w-100 fw-bold shadow-
            sm">
            <i class="bi bi-trash"></i> Delete
        </button>
    </div>
</div>

</div>
</div>
</div>
<?php endforeach; ?>
</div>
</form>
<?php else: ?>
    <!--If user has no sightings-->
    <div class="alert alert-info text-center fw-semibold shadow-sm">
        You have not recorded any sightings yet.
    </div>
<?php endif; ?>
</div>
</div>
</div>
</div>
</div>
</div>
</div>
<?php require once('Views/template/footer.phtml'); ?>

```

mypet.php

```
<?php
// Start a session
session_start();
// Inheritance the required database model classes
require_once('Models/Database.php');
require_once('Models/PetDataSets.php');

// Check if a user is not logged in
if (empty($_SESSION['user_id'])) {
    $view = new stdClass();
    // Show error on the login form when not logged in
    $view->errorMessage = "You must log in to add and manage pet";
    // Load the login view
    require_once('Views/login.phtml');
    exit();
}

// Get logged-in user ID (always cast to int for safety)
$user_id = (int)$SESSION['user_id'];
```

```

// Create model instance
$petsDataSet = new PetDataSets();

// View setup
$view = new stdClass();
$view->title = "My Pets";
$view->successMessage = '';
$view->errorMessage = '';

/**
 * Validate pet data from form input
 * Makes sure name, breed, color, and description meet length/pattern rules
 */
function validateMypetData($input): array {
    // Trim and sanitize input
    $name = trim($input['name'] ?? '');
    $breed = trim($input['breed'] ?? '');
    $color = trim($input['color'] ?? '');
    $description = trim($input['description'] ?? '');

    $errors = [];

    // Validate lengths only, keeping input flexible and secure
    if (mb_strlen($name) > 20)
        $errors[] = "Pet name must be up to 20 characters.";
    elseif (mb_strlen($breed) > 50)
        $errors[] = "Breed name must be up to 50 characters.";
    elseif (mb_strlen($color) > 20)
        $errors[] = "Color must be up to 20 characters.";
    elseif (mb_strlen($description) > 200)
        $errors[] = "Description must be up to 200 characters.";

    // Escape for output safety (prevents XSS), converts special characters
    // into safe html entities, ENT_QUOTES encodes both single and double
    quotes
    $name = htmlspecialchars($name, ENT_QUOTES, 'UTF-8');
    $breed = htmlspecialchars($breed, ENT_QUOTES, 'UTF-8');
    $color = htmlspecialchars($color, ENT_QUOTES, 'UTF-8');
    $description = htmlspecialchars($description, ENT_QUOTES, 'UTF-8');

    return [$name, $breed, $color, $description, $errors];
}

/**
 * Handles pet image upload
 * Verifies allowed extensions and moves file to /images/pets
 * @throws Exception
 */
function handlePetPhotoUpload(string $fieldName): ?string {
    // No file uploaded
    if (!isset($_FILES[$fieldName]) ||
        $_FILES[$fieldName]['error'] !== UPLOAD_ERR_OK) {
        return null;
    }

    // For this project, we save uploaded images in a local directory.
    // In a real-world app, images would typically be stored in cloud storage.
    $uploadDir = __DIR__ . '/images/pets';
    if (!is_dir($uploadDir)) mkdir($uploadDir, 0777, true);

    // Get temp path, extract extension, allow only safe formats

```

```

$fileTmp = $_FILES[$fieldName]['tmp_name'];
$fileExt = strtolower(
    pathinfo($_FILES[$fieldName]['name'], PATHINFO_EXTENSION)
);
$allowed = ['jpg', 'jpeg', 'png', 'gif'];

// Only allow specified formats
if (!in_array($fileExt, $allowed)) {
    throw new Exception("Invalid file type. Only .gif, .jpg, .jpeg, .png
allowed");
}

// Generate unique file name to avoid collisions
$newFileName = uniqid('pet_', true) . '.' . $fileExt;
$targetPath = "$uploadDir/$newFileName";

// Move file to target directory
if (!move_uploaded_file($fileTmp, $targetPath)) {
    throw new Exception("Error moving uploaded file.");
}

// Return relative path to store in DB
return "images/pets/$newFileName";
}

// Handle all form submissions such as addPet, update, delete
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $action = '';

    // Detect which button was clicked
    if (isset($_POST['addPet'])) $action = 'create';
    elseif (isset($_POST['update'])) $action = 'update';
    elseif (isset($_POST['deletePet'])) $action = 'delete';

    try {
        switch ($action) {
            // Create new pet
            case 'create':
                // Validate before creating in db
                [$name, $breed, $color, $description, $errors] =
                    validateMypetData($_POST);
                $species = htmlspecialchars(trim($_POST['species'] ?? ''));

                if (!empty($errors)) {
                    $view->errorMessage = implode("<br>", $errors);
                    break;
                }

                // Upload pet photo
                $photo_url = handlePetPhotoUpload('petphoto');
                if (!$photo_url)
                    throw new Exception("No photo uploaded or upload error.");

                // Add pet to db
                $petsDataSet->addPet(
                    $user_id, $name, $species, $breed, $color, $description, $photo_url
                );
                $view->successMessage = "Pet added successfully!";
                break;

                // Update existing pet

```

```

case 'update':
    $petId = (int)($_POST['update'] ?? 0);
    if ($petId <= 0 || !isset($_POST['pets'][$petId])) {
        throw new Exception("Invalid pet data.");
    }

    $petInput = $_POST['pets'][$petId];
    [$name, $breed, $color, $description, $errors] =
        validateMypetData($petInput);
    $species = htmlspecialchars(trim($petInput['species'] ?? ''));

    if (!empty($errors)) {
        $view->errorMessage = implode("<br>", $errors);
        break;
    }

    // Build data array for update
    $data = [
        'name' => $name,
        'breed' => $breed,
        'color' => $color,
        'description' => $description,
        'species' => $species
    ];

    // If new photo uploaded, replace old one
    $photoField = 'photo_' . $petId;
    $photo_url = handlePetPhotoUpload($photoField);
    if ($photo_url) $data['photo_url'] = $photo_url;

    // Update database
    $updated = $petsDataSet->updatePet($petId, $user_id, $data);
    $view->successMessage = $updated
        ? "Pet updated successfully."
        : "Pet update failed.";
    break;

// Delete pet data
case 'delete':
    $petId = (int)($_POST['deletePet'] ?? 0);
    if ($petId <= 0) throw new Exception("Invalid pet ID.");

    $deleted = $petsDataSet->deletePet($petId, $user_id);
    $view->successMessage = $deleted
        ? "Pet deleted successfully."
        : "Could not delete pet.";
    break;

default:
    // No form action triggered
    break;
}
} catch (Exception $e) {
    $view->errorMessage = $e->getMessage();
}
}

// Always load user's pets after actions
$view->pets = $petsDataSet->getPetByUserID($user_id);

```

```
// Load the pet view
require_once('Views/mypet.phtml');
```

mypet.phtml associated with mypet.php

```
<?php
// Load the header
require('Views/template/header.phtml');
?>

<!--Check if any user is logged in-->
<?php if (isset($_SESSION['loggedin']) && $_SESSION['loggedin'] === true):
?>

    <!-- If add pet or manage pet update info is successful show the relevant
    message -->
    <?php if (!empty($view->successMessage)): ?>
        <div class="col-lg-10 mx-auto justify-content-center mt-5">
            <div class="alert alert-success text-center shadow-sm rounded w-100 w-
md-75 w-lg-50">
                <?= htmlspecialchars($view->successMessage); ?>
            </div>
        </div>
    <?php endif; ?>

    <!--If not successful, for example an owner uploaded an unsupported photo
    format show what went wrong -->
    <?php if (!empty($view->errorMessage)): ?>
        <div class="col-lg-10 mx-auto justify-content-center mt-5">
            <div class="alert alert-danger text-center shadow-sm rounded w-100 w-md-
75 w-lg-50">
                <?= htmlspecialchars($view->errorMessage); ?>
            </div>
        </div>
    <?php endif; ?>

    <!--Owner can add pet data on database using below form-->
    <div class="container py-4 mt-5">
        <div class="row">
            <div class="col-lg-10 mx-auto">
                <div class="card shadow-sm border-0 bg-light rounded-3">
                    <div class="card-body p-4">
                        <h2 class="text-center mb-5 display-6 text-primary fw-semibold">Add A
New Pet</h2>
                        <p class="text-left"><span class="text-danger">* Indicates Required
Field</span></p>

                        <form action="/mypet.php" method="post" enctype="multipart/form-
data">
                            <div class="row g-3">

                                <!-- Pet Name required field -->
                                <div class="col-md-6 col-lg-3">
                                    <label for="petName" class="form-label small fw-semibold">
Name <span class="text-danger">*</span>
                                    </label>
                                    <input type="text" id="petName" name="name"
class="form-control shadow-sm"
placeholder="Pet Name" required>
                                </div>
                            </div>
                        </form>
                    </div>
                </div>
            </div>
        </div>
    </div>
```

```

</div>

<!-- Pet species (required field) -->
<div class="col-md-6 col-lg-3">
  <label for="species" class="form-label small fw-semibold">
    Species <span class="text-danger">*</span>
  </label>
  <select id="species" name="species"
    class="form-select shadow-sm" required>
    <option value="" disabled selected>Select Species</option>
    <option value="Dog">Dog</option>
    <option value="Cat">Cat</option>
    <option value="Bird">Bird</option>
  </select>
</div>

<!-- Breed optional field -->
<div class="col-md-6 col-lg-3">
  <label for="breed" class="form-label small fw-semibold">Breed</label>
  <input type="text" id="breed" name="breed"
    class="form-control shadow-sm"
    placeholder="Breed">
</div>

<!-- Color required field -->
<div class="col-md-6 col-lg-3">
  <label for="color" class="form-label small fw-semibold">
    Color <span class="text-danger">*</span>
  </label>
  <input type="text" id="color" name="color"
    class="form-control shadow-sm"
    placeholder="Color" required>
</div>

<!-- Description optional field -->
<div class="col-md-8">
  <label for="description" class="form-label small fw-semibold">
    Short Description
  </label>
  <textarea id="description" name="description"
    class="form-control shadow-sm" rows="1"
    placeholder="Briefly describe your pet
traits."></textarea>
</div>

<!-- Upload Image required field -->
<div class="col-md-4">
  <label for="photo" class="form-label small fw-semibold">
    Upload Pet Image
  </label>
  <input type="file" id="photo" name="petphoto"
    class="form-control shadow-sm"
    accept=".gif,.jpg,.jpeg,.png" required>
</div>
</div>

<!-- Submit and add a pet -->
<div class="text-left mt-4">
  <button type="submit" name="addPet"
    class="btn btn-primary px-5 py-3 fw-bold shadow-sm">

```

```

        Add Pet
    </button>
</div>
</form>
</div>
</div>
</div>
</div>
</div>

<!--Owner can read, update and delete their pet data using this form -->
<div class="container pb-5 mb-5 mt-5">
    <div class="row">
        <div class="col-lg-10 mx-auto">
            <div class="card shadow-sm border-0 bg-light rounded-3">
                <div class="card-body p-4">

                    <form action="/mypet.php" method="post" enctype="multipart/form-
                    data">
                        <h2 class="text-center mb-5 display-6 text-primary fw-semibold">
                            Manage Your Pets
                        </h2>

                        <div class="row mx-2 gy-4">
                            <!--If a logged-in user has pets show the pets they own -->
                            <?php if (!empty($view->pets)): ?>
                                <!--Get all existing pet data -->
                                <?php foreach ($view->pets as $pet): ?>
                                    <div class="col-12">
                                        <div class="card shadow-sm border-0 bg-white rounded-3">
                                            <div class="card-body px-3 py-4">
                                                <div class="row gy-3 gx-4 align-items-start">

                                                    <!--Get the current image of each pet-->
                                                    <div class="col-12 col-md-3 text-center border-end">
                                                        <label class="form-label fw-bold text-muted">Current
                                                            Image</label>
                                                        <!--If the pet has an image on database fetch that image-->
                                                        <?php if ($pet->getPetImages()): ?>
                                                            
                                                        <?php else: ?>
                                                            <!--Else show no image has been uploaded -->
                                                            <div class="border border-info rounded p-4 text-info bg-
                                                                white
                                                                small mb-3 shadow-sm">
                                                                No image uploaded
                                                            </div>
                                                        <?php endif; ?>

                                                    <!--User can change the existing image of their pet by click
                                                        on
                                                        this functionality -->
                                                    <label for="photo_<?= $pet->getId() ?>" class="form-label
                                                        small
                                                        fw-semibold">
                                                        Change Pet Image
                                                    </label>

```

```

        <input type="file" name="photo_<?= $pet->getId() ?>"
              class="form-control form-control-sm shadow-sm"
              accept=".gif,.jpg,.jpeg,.png">
    </div>

    <!--Pet details form-->
    <div class="col-12 col-md-9">
        <h4 class="card-title mb-3 border-bottom pb-2 fw-semibold
        text-primary">
            <?= htmlspecialchars($pet->getName()) ?>
        </h4>

        <div class="row g-3">
            <!--Get pet name by id and allow owner to change the name-->
            <div class="col-md-6 col-lg-3">
                <label class="form-label small text-muted">Name</label>
                <input type="text" name="pets[<?= $pet->getId() ?>][name]"
                    value="<?= htmlspecialchars($pet->getName()) ?>"
                    class="form-control shadow-sm" placeholder="Pet
                    Name">
            </div>

            <!--Get pet breed and allow to change it -->
            <div class="col-md-6 col-lg-3">
                <label class="form-label small text-muted">Breed</label>
                <input type="text" name="pets[<?= $pet->getId()
                ?>][breed]"
                    value="<?= htmlspecialchars($pet->getBreed()) ?>"
                    class="form-control shadow-sm" placeholder="Breed">
            </div>

            <!--Get pet color and allow to change it -->
            <div class="col-md-6 col-lg-3">
                <label class="form-label small text-muted">Color</label>
                <input type="text" name="pets[<?= $pet->getId()
                ?>][color]"
                    value="<?= htmlspecialchars($pet->getColor()) ?>"
                    class="form-control shadow-sm" placeholder="Color">
            </div>

            <!--Get pet species and allow owner to change the species-->
            <div class="col-md-6 col-lg-3">
                <label class="form-label small text-muted">Species</label>
                <select name="pets[<?= $pet->getId() ?>][species]"
                    class="form-select shadow-sm">
                    <option value="Dog"
                        <?= ($pet->getSpecies() === 'Dog') ? 'selected' :
                        '' ?>>
                        Dog
                    </option>
                    <option value="Cat"
                        <?= ($pet->getSpecies() === 'Cat') ? 'selected' :
                        '' ?>>
                        Cat
                    </option>
                    <option value="Bird"
                        <?= ($pet->getSpecies() === 'Bird') ? 'selected'
                        : '' ?>>
                        Bird
                    </option>

```

```

        </select>
    </div>

    <!--Get current description then allow to change it-->
    <div class="col-md-9">
        <label class="form-label small text-
            muted">Description</label>
        <textarea name="pets[<?=$pet->getId() ?>][description]"
            class="form-control shadow-sm" rows="1"
            placeholder="Tell us about your pet..."
            <?=$htmlspecialchars($pet->getDescription())
                ?></textarea>

    </div>

    <!--Get lost and allow to change the status-->
    <div class="col-md-3">
        <label class="form-label small text-muted">Status</label>
        <select name="pets[<?=$pet->getId() ?>][status]"
            class="form-select shadow-sm">
            <option value="lost"
                <?=$pet->getStatus() === 'lost' ? 'selected' :
                    '' ?>>

                Lost
            </option>
            <option value="found"
                <?=$pet->getStatus() === 'found' ? 'selected'
                    : '' ?>>

                Found
            </option>
        </select>
    </div>
</div>

<!--Update pet info after editing by below process-->
<div class="row g-3 mt-4">
    <div class="col-md-6 col-lg-4">
        <!--After changing the data user can click on update pet
            info to update the database-->
        <button type="submit" name="update"
            value="<?=$pet->getId() ?>"
            class="btn btn-success w-100 fw-bold shadow-sm">
            <i class="bi bi-pencil-square"></i> Update Pet Info
        </button>
    </div>

    <!--Allow to delete the pet information by getting the pet
        id-->
    <div class="col-md-6 col-lg-4">
        <button type="button"
            class="btn btn-outline-danger w-100 fw-bold
                shadow-sm"
            data-bs-toggle="modal"
            data-bs-target="#deleteConfirmModal"
            data-pet-id="<?=$pet->getId() ?>"
            data-pet-name="<?=$htmlspecialchars($pet->
                getName()) ?>">
            <i class="bi bi-trash"></i> Delete
        </button>
    </div>
</div>
</div>

```

```

        </div>
    </div>
</div>
</div>
<?php endforeach; ?>

<!--If an owner has not registered a pet yet, show them below
message.-->
<?php else: ?>
    <div class="col-12">
        <div class="alert alert-info text-center fw-semibold shadow-sm">
            You have no pets added yet.<br>
            Use the "Add New Pet" feature to get started!
        </div>
    </div>
<?php endif; ?>
</div>
</form>
</div>
</div>
</div>
</div>
</div>

<!-- Delete Confirmation Modal utilising bootstrap is implemented which is
used to
confirm delete to avoid deleting important information by mis click
Note: This is an optional implementation and I have used a small
JavaScript snippet.
-->
<div class="modal fade" id="deleteConfirmModal" aria-hidden="true">
    <div class="modal-dialog modal-dialog-centered">
        <div class="modal-content border-0 shadow-lg rounded-3">
            <div class="modal-header bg-danger text-white">
                <h5 class="modal-title">
                    <i class="bi bi-exclamation-triangle"></i> Confirm Deletion
                </h5>
                <button type="button" class="btn-close btn-close-white"
                    data-bs-dismiss="modal"></button>
            </div>
            <div class="modal-body text-center fs-5 py-4">
                Are you sure you want to delete
                <!--get the pet name and show it in bold danger to verify
                user that they are sure-->
                <span id="deletePetName" class="fw-bold text-danger"></span>
                and all related sightings?
            </div>
            <div class="modal-footer justify-content-center border-0 pb-4">
                <form id="deletePetForm" method="post" action="mypet.php">
                    <input type="hidden" name="deletePet" id="deletePetId">

                    <!--Allow owner to cancel to delete-->
                    <button type="button"
                        class="btn btn-secondary fw-semibold shadow-sm"
                        data-bs-dismiss="modal">Cancel</button>

                    <!--when pressed on Yes, Delete the form will then delete
                    that pet information-->
                    <button type="submit"
                        class="btn btn-danger px-4 fw-semibold shadow-sm">
                        Yes, Delete

```

```

        </button>
    </form>
</div>
</div>
</div>
</div>

<!-- JavaScript snippet to the modal and show relevant information-->
<script>
    /*Get the delete confirmation modal element*/
    const deleteModal = document.getElementById('deleteConfirmModal');
    /* Add even listener when the modal is about to be shown */
    deleteModal.addEventListener('show.bs.modal', event => {
        /*Button to triggers the modal */
        const button = event.relatedTarget;
        /* Pass the pet id into the form which then will be handled by php
        script to delete the pet information*/
        document.getElementById('deletePetId').value =
            button.getAttribute('data-pet-id');
        /* Show the pet name on the modal */
        document.getElementById('deletePetName').textContent =
            button.getAttribute('data-pet-name');
    });
</script>

<?php else: ?>
    <div class="container text-center my-5">
        <div class="alert alert-warning shadow-sm">
            You must be logged in to view your pets.
        </div>
        <a href="/login.php" class="btn btn-primary fw-semibold shadow-sm">
            Login
        </a>
    </div>
<?php endif; ?>

<?php require('Views/template/footer.phtml'); ?>

```

E) Custom CSS files

1. index.css

```
/* Page container so when page is smaller than view height footer doesn't
go up*/
.page-container {
  display: flex;
  flex-direction: column;
  min-height: 100vh;
}

/* It allows us to push the footer on the bottom of the view */
main {
  flex: 1;
}

main > .container {
  padding: 60px 15px 0;
}

/* =====: Header custom css :=====*/

/* Set the logo to desirable style */
.logo {
  height: 40px;
  width: 40px;
}

.nav-link {
  color: #cccccc;
}

.nav-link:hover {
  text-decoration: underline;
}

.navbar-brand {
  font-weight: bold;
}

#title {
  margin-top: 12px;
  background-color: #fff;
  color: #000;
}

.user-details {
  color: #cccccc;
}

#content {
  background-color: #fff;
  border-top: solid 6px #f00;
}

.form-select {
  font-size: 14px;
  width: 100%;
}
```

```

}

/* =====: Custom css for hero section :===== */
.hero-section {
  background: linear-gradient(135deg, #343a40 0%, #495057 100%);
  color: white;
  padding: 80px 0 60px 0;
  margin-top: 0;
  text-align: center;
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
}

.hero-title {
  font-size: 3rem;
  font-weight: 700;
  margin-bottom: 20px;
  text-shadow: 2px 2px 4px rgba(0, 0, 0, 0.3);
}

.hero-subtitle {
  font-size: 1.3rem;
  font-weight: 300;
  margin-bottom: 30px;
  color: #f8f9fa;
}

.hero-buttons {
  display: flex;
  gap: 15px;
  justify-content: center;
  flex-wrap: wrap;
}

.hero-btn {
  padding: 12px 30px;
  font-size: 1.1rem;
  font-weight: 600;
  border-radius: 8px;
  transition: all 0.3s ease;
  text-decoration: none;
  display: inline-block;
}

.hero-btn-primary {
  background-color: #198754;
  color: white;
  border: 2px solid #198754;
}

.hero-btn-primary:hover {
  background-color: #157347;
  color: white;
  transform: translateY(-2px);
  box-shadow: 0 4px 12px rgba(25, 135, 84, 0.4);
}

.hero-btn-secondary {
  background-color: transparent;
  color: white;
  border: 2px solid white;
}

```

```

}

.hero-btn-secondary:hover {
  background-color: white;
  color: #343a40;
  transform: translateY(-2px);
  box-shadow: 0 4px 12px rgba(255, 255, 255, 0.3);
}

/* ===== Custom css for feature section on home page :===== */
.features-section {
  padding: 60px 0;
}

.section-title {
  text-align: center;
  font-size: 2.5rem;
  font-weight: 700;
  color: #343a40;
  margin-bottom: 50px;
}

.feature-card {
  border: none;
  border-radius: 15px;
  overflow: hidden;
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
  transition: all 0.3s ease;
  height: 100%;
}

.feature-card:hover {
  transform: translateY(-8px);
  box-shadow: 0 8px 24px rgba(0, 0, 0, 0.2);
}

.feature-card img {
  height: 250px;
  object-fit: cover;
  transition: all 0.3s ease;
}

.feature-card:hover img {
  transform: scale(1.05);
}

.feature-card-body {
  padding: 25px;
}

.feature-card-title {
  font-size: 1.5rem;
  font-weight: 700;
  color: #343a40;
  margin-bottom: 15px;
}

.feature-card-text {
  color: #6c757d;
  line-height: 1.6;
}

```

```

        margin-bottom: 20px;
    }

    .feature-btn {
        background: linear-gradient(135deg, #343a40 0%, #495057 100%);
        color: white;
        border: none;
        padding: 10px 25px;
        border-radius: 8px;
        font-weight: 600;
        transition: all 0.3s ease;
        text-decoration: none;
        display: inline-block;
    }

    .feature-btn:hover {
        background: linear-gradient(135deg, #495057 0%, #6c757d 100%);
        color: white;
        transform: translateY(-2px);
        box-shadow: 0 4px 12px rgba(52, 58, 64, 0.3);
    }

    /* =====: Custom css for demo stats :===== */
    .stats-section {
        background-color: #f8f9fa;
        padding: 50px 0;
        text-align: center;
    }

    .stat-item {
        padding: 20px;
    }

    .stat-number {
        font-size: 3rem;
        font-weight: 700;
        color: #198754;
    }

    .stat-label {
        font-size: 1.1rem;
        color: #6c757d;
        margin-top: 10px;
    }

    /* =====: Custom css for footer: =====*/
    .footer {
        margin-top: 20px;
        text-align: center;
        background-color: #333333;
        color: white;
    }

    .footer > p {
        color: white;
    }

    .footer-intro {
        color: #cccccc;
        font-size: 14px;
    }

```

```

}

.footer a:hover {
    color: #cccccc;
}

.form-control:focus, .form-select:focus {
    box-shadow: 0 0 0 0.1rem rgba(25, 135, 84, 0.25); /* soft green glow */
    border-color: #198754;
    outline: none;
    border: 0px;
}

/* Modern Footer Styling */
.modern-footer {
    background: linear-gradient(135deg, #212529 0%, #343a40 100%);
    color: #f8f9fa;
    padding: 50px 0 20px 0;
    margin-top: auto;
}

.footer-content {
    padding-bottom: 30px;
    border-bottom: 1px solid #495057;
}

.footer-section {
    margin-bottom: 30px;
}

.footer-title {
    font-size: 1.3rem;
    font-weight: 700;
    color: #fff;
    margin-bottom: 20px;
    position: relative;
    padding-bottom: 10px;
}

.footer-title::after {
    content: '';
    position: absolute;
    left: 0;
    bottom: 0;
    width: 50px;
    height: 3px;
    background-color: #198754;
    border-radius: 2px;
}

.footer-logo {
    display: flex;
    align-items: center;
    gap: 10px;
    margin-bottom: 15px;
}

.footer-logo img {
    height: 40px;
    width: 40px;
}

```

```

.footer-logo-text {
  font-size: 1.5rem;
  font-weight: 700;
  color: #fff;
}

.footer-description {
  color: #adb5bd;
  line-height: 1.6;
  margin-bottom: 20px;
}

.footer-links {
  list-style: none;
  padding: 0;
}

.footer-links li {
  margin-bottom: 12px;
}

.footer-links a {
  color: #adb5bd;
  text-decoration: none;
  transition: all 0.3s ease;
  display: inline-block;
}

.footer-links a:hover {
  color: #198754;
  padding-left: 5px;
}

.footer-social {
  display: flex;
  gap: 15px;
  margin-top: 20px;
}

.social-icon {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 40px;
  height: 40px;
  background-color: #343a40;
  border-radius: 50%;
  color: #f8f9fa;
  text-decoration: none;
  transition: all 0.3s ease;
  font-size: 1.1rem;
}

.social-icon:hover {
  background-color: #198754;
  color: white;
  transform: translateY(-3px);
  box-shadow: 0 4px 8px rgba(25, 135, 84, 0.3);
}

```

```

.footer-contact-info {
  color: #adb5bd;
  margin-bottom: 10px;
}

.footer-contact-info strong {
  color: #f8f9fa;
}

.footer-bottom {
  padding-top: 25px;
  text-align: center;
}

.footer-copyright {
  color: #6c757d;
  font-size: 0.9rem;
  margin: 0;
}

.footer-bottom-links {
  display: flex;
  justify-content: center;
  gap: 20px;
  margin-top: 10px;
  flex-wrap: wrap;
}

.footer-bottom-links a {
  color: #6c757d;
  text-decoration: none;
  font-size: 0.9rem;
  transition: color 0.3s ease;
}

.footer-bottom-links a:hover {
  color: #198754;
}

/* =====: Media Query for small and medium devices :=====
*/
@media (max-width: 768px) {

  .login-btn {
    margin: 20px 0px;
  }

  .user-details {
    margin: 8px 15px;
    color: #cccccc;
  }

  .navbar-toggler:focus {
    outline: none;
    box-shadow: none;
    border: none;
  }

  .user-details {
    margin: 15px 0px;
  }
}

```

```

        color: #cccccc;
    }

    .logout-btn {
        margin-bottom: 15px;
    }

    .search {
        margin-bottom: 20px;
    }

    /* Hero section */
    .hero-title {
        font-size: 2rem;
    }

    .hero-subtitle {
        font-size: 1.1rem;
    }

    .section-title {
        font-size: 2rem;
    }

    .feature-card img {
        height: 200px;
    }

    /* Footer Section */
    .footer-title::after {
        width: 40px;
    }

    .footer-social {
        justify-content: center;
    }

    .modern-footer {
        padding: 40px 0 20px 0;
    }
}

/* =====: Media query for lg and bigger devices
:===== */
@media (min-width: 992px) {

    .user-details {
        margin: 8px 15px;
        color: #cccccc;
    }

    .logout-btn {
        margin-right: 0;
        margin-left: 10px;
    }

    .logout-btn .btn {
        padding: 8px 20px;
        font-size: 14px;
    }
}

```

```

        .scrollable-dropdown {
            max-height: 100px;
            overflow-y: auto;
        }
    }
}

```

2. login.css

```

/* =====: Custom CSS for login form :===== */
.login-form{
    height: 80vh;
    margin-top:50px;
}

.login-card {
    box-shadow: 0 8px 24px rgba(0, 0, 0, 0.15);
    border-radius: 15px;
    padding: 30px;
    background: white;
    margin-top: 60px;
}

.login-title {
    font-size: 1.8rem;
    font-weight: 700;
    color: #343a40;
    text-align: center;
    margin-bottom: 30px;
}

/* =====: Custom CSS for logout form and welcome message :===== */
.welcome-card {
    background: linear-gradient(135deg, #fff 0%, #f8f9fa 100%);
    border-radius: 15px;
    box-shadow: 0 4px 20px rgba(0, 0, 0, 0.1);
    padding: 40px;
    margin-top: 60px;
    text-align: center;
}

.welcome-title {
    font-size: 2rem;
    font-weight: 700;
    color: #343a40;
    margin-bottom: 15px;
}

.welcome-subtitle {
    color: #6c757d;
    margin-bottom: 30px;
}

.logout-btn {
    border: none;
    font-weight: 600;
    border-radius: 8px;
}

```

```

        transition: all 0.3s ease;
    }

.logout-btn:hover {
    transform: translateY(-2px);
    box-shadow: 0 4px 12px rgba(220, 53, 69, 0.4);
}

/* =====: Custom css for quick links on welcome card :===== */
.quick-links {
    display: flex;
    gap: 15px;
    justify-content: center;
    margin-top: 20px;
}

.quick-link-btn {
    background-color: #343a40;
    color: white;
    padding: 10px 20px;
    border-radius: 8px;
    text-decoration: none;
    font-weight: 600;
    transition: all 0.3s ease;
}

.quick-link-btn:hover {
    background-color: #198754;
    color: white;
    transform: translateY(-2px);
}

```

3. sightings.css

```

/* =====: Custom CSS for pagination :===== */
.search-highlight {
    background-color: #ffeb3b;
    padding: 2px 4px;
    border-radius: 3px;
    font-weight: bold;
    color: #000;
}

/* Dark Theme Pagination */
.pagination {
    gap: 8px;
    margin-top: 2rem;
    margin-bottom: 2rem;
}

.pagination .page-item {
    margin: 0;
}

.pagination .page-link {
    background-color: #343a40;
}

```

```

border: 2px solid #495057;
border-radius: 8px;
color: #fff;
font-weight: 500;
padding: 10px 16px;
transition: all 0.3s ease;
box-shadow: 0 2px 6px rgba(0, 0, 0, 0.3);
min-width: 45px;
text-align: center;
}

.pagination .page-link:hover {
background-color: #495057;
color: #fff;
border-color: #6c757d;
transform: translateY(-2px);
box-shadow: 0 4px 12px rgba(0, 0, 0, 0.5);
}

.pagination .page-item.active .page-link {
background-color: #198754;
border-color: #198754;
color: white;
font-weight: 600;
box-shadow: 0 4px 12px rgba(25, 135, 84, 0.5);
}

.pagination .page-item.active .page-link:hover {
background-color: #157347;
border-color: #157347;
}

.pagination .page-item.disabled .page-link {
background-color: #212529;
border-color: #343a40;
color: #6c757d;
cursor: not-allowed;
opacity: 0.6;
}

.pagination .page-item.disabled .page-link:hover {
background-color: #212529;
border-color: #343a40;
color: #6c757d;
transform: none;
box-shadow: 0 2px 6px rgba(0, 0, 0, 0.3);
}

/* Previous/Next buttons special styling */
.pagination .page-item:first-child .page-link,
.pagination .page-item:last-child .page-link {
font-weight: 600;
padding: 10px 20px;
}

/* =====: Media query for small devices :===== */
@media (max-width: 576px) {
.pagination .page-link {
padding: 8px 12px;
font-size: 14px;
min-width: 38px;
}

```

```

    }

    .pagination .page-item:first-child .page-link,
    .pagination .page-item:last-child .page-link {
        padding: 8px 14px;
    }
}

/* Modern Sightings Card Styling */
.sighting-card {
    background: white;
    border-radius: 12px;
    box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
    margin-bottom: 20px;
    overflow: hidden;
    transition: all 0.3s ease;
    border: 1px solid #e0e0e0;
}

.sighting-card:hover {
    box-shadow: 0 4px 16px rgba(0, 0, 0, 0.15);
    transform: translateY(-2px);
}

.sighting-card-header {
    background: linear-gradient(135deg, #343a40 0%, #495057 100%);
    color: white;
    padding: 15px 20px;
    font-weight: 600;
    display: flex;
    justify-content: space-between;
    align-items: center;
}

.sighting-card-body {
    padding: 20px;
}

.sighting-image {
    width: 100%;
    max-width: 200px;
    height: auto;
    border-radius: 10px;
    box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
    object-fit: cover;
}

.sighting-info {
    display: flex;
    flex-direction: column;
    gap: 10px;
}

.info-row {
    display: flex;
    align-items: flex-start;
    gap: 10px;
}

.info-label {
    font-weight: 600;
}

```

```

        color: #495057;
        min-width: 100px;
        font-size: 14px;
    }

    .info-value {
        color: #212529;
        flex: 1;
        font-size: 14px;
    }

    .status-badge {
        display: inline-block;
        padding: 6px 12px;
        border-radius: 20px;
        font-size: 12px;
        font-weight: 600;
        text-transform: uppercase;
    }

    /* If the status is lost show it with red background */
    .status-badge.lost {
        background-color: #dc3545;
        color: white;
    }

    /*If status is found show it with green background */
    .status-badge.found {
        background-color: #198754;
        color: white;
    }

    .comment-box {
        background-color: #f8f9fa;
        padding: 12px;
        border-radius: 8px;
        border-left: 4px solid #198754;
        font-style: italic;
        color: #495057;
        margin-top: 5px;
    }

    .timestamp {
        color: #6c757d;
        font-size: 13px;
        font-style: italic;
    }

    .coordinates {
        font-family: 'Courier New', monospace;
        background-color: #e9ecef;
        padding: 4px 8px;
        border-radius: 4px;
        font-size: 12px;
    }

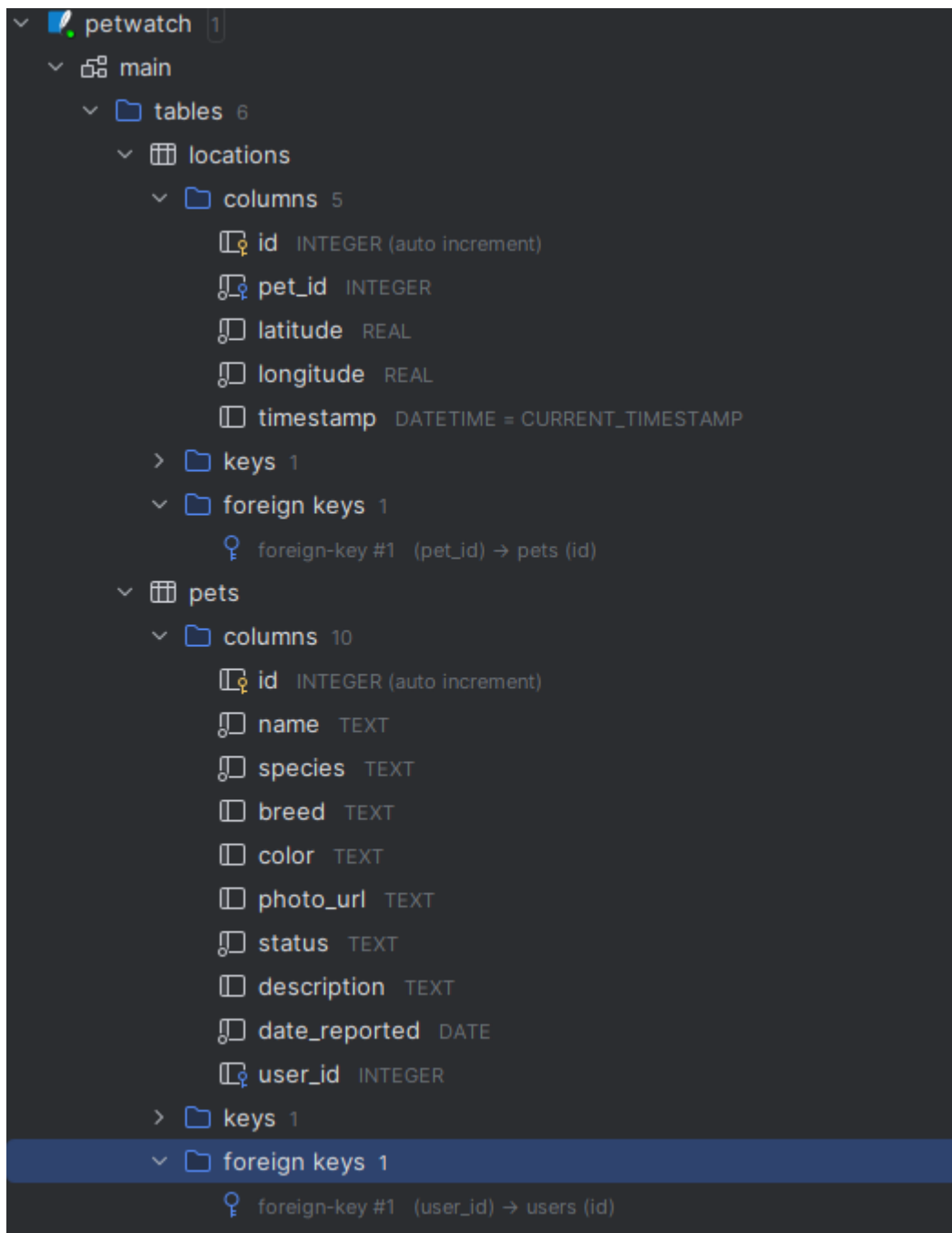
    /* =====: Media Query for medium devices :===== */
    @media (max-width: 768px) {
        .sighting-card-body {
            padding: 15px;
        }
    }

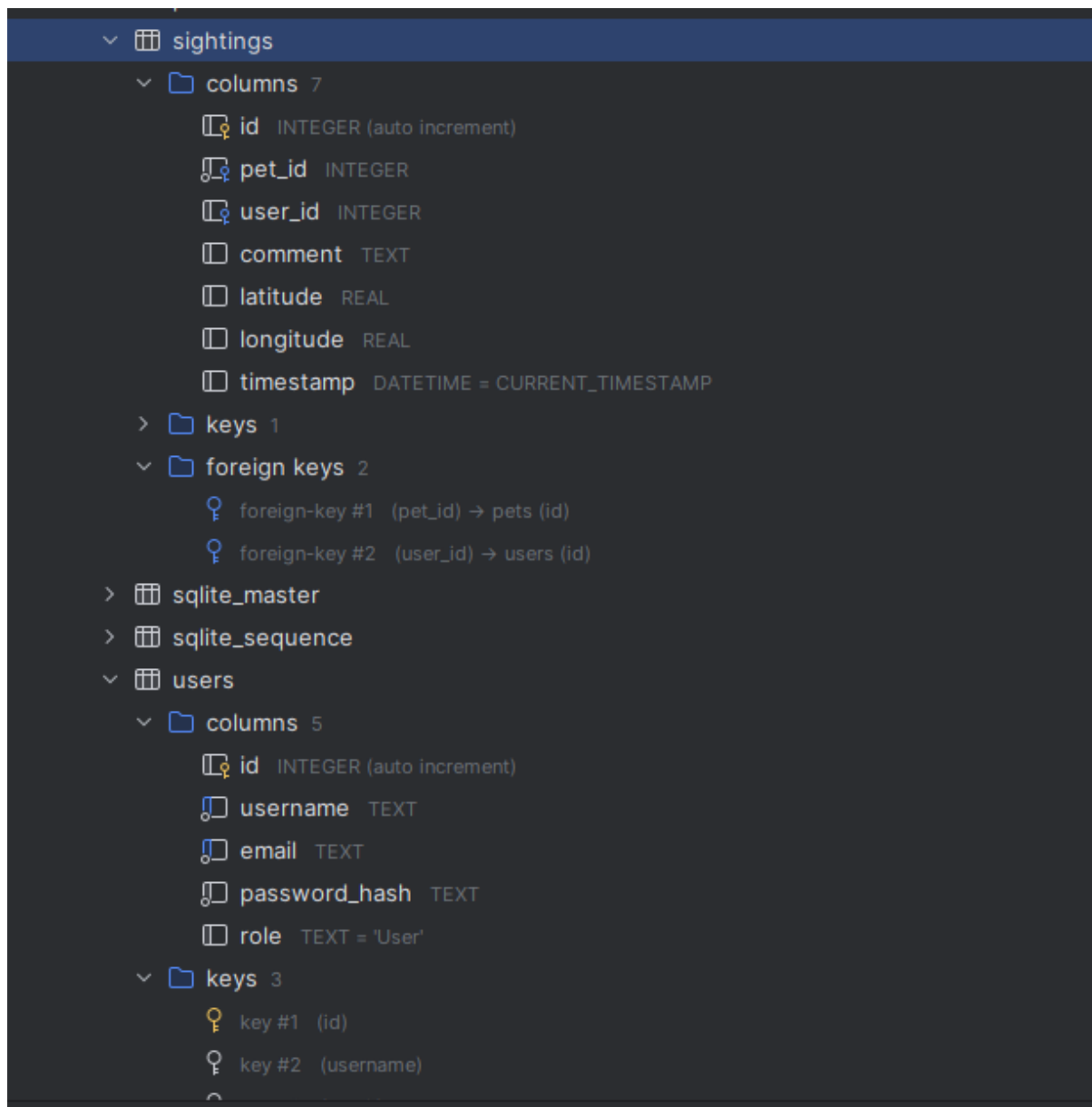
```

```
.sighting-image {  
  max-width: 100%;  
  margin-bottom: 15px;  
}  
  
.info-label {  
  min-width: 80px;  
}  
}
```

F) Database Records

Tables





Data snapshot for users table (total 210)

	id	username	email	password_hash	role
1	1	Lee	lee@example.com	\$2y\$10\$2guAQV4XTa.MTAUTQvHypuB...	Owner
2	2	Zara	zara@example.com	\$2y\$10\$uAQFV.QY6YJ5zJPqj.Xo.uH...	User
3	3	Sujon	sujon@example.com	\$2y\$10\$e8YRhr1oUTctHVaCd0p2Q.n...	Owner
4	4	Soren	sor@example.com	\$2y\$10\$QKd2ZNLmG56rBArYQ0dbX.E...	Owner
5	5	Anya	any@example.com	\$2y\$10\$V07tIzMvKfImDtZ/i8u9keZ...	Owner
6	6	Jaxon	jax@example.com	\$2y\$10\$LwwUiy5x4Ixx0IB/S9sPa0....	Owner
7	7	Tamsin	tam@example.com	\$2y\$10\$/uZS8mw8CDytcnblCWPF0x...	Owner
8	8	Elias	eli@example.com	\$2y\$10\$en5KKfbAcy.DRnALyRUHguJ...	Owner
9	9	Petra	pet@example.com	\$2y\$10\$ezFCinDxCbcE5TTkm8NL7eL...	Owner
10	10	Orion	ori@example.com	\$2y\$10\$iaCjhF6cRfW0./Rvi3Y110n...	Owner
11	11	Zelia	zel@example.com	\$2y\$10\$DnEGMgzx8F/ZET.0Vz69Tu....	Owner
12	12	Kaelen	kae@example.com	\$2y\$10\$jT/soDMzH3BDwxaPLAIqb0Q...	Owner
13	13	Shula	shu@example.com	\$2y\$10\$mz36KvnI6DYT.xJBmw54suH...	Owner
14	14	Dorian	dor@example.com	\$2y\$10\$Az80i266njPaATEEKPMtv0T...	Owner

197	197	user197	user197@example.com	\$2y\$10\$0wiQ6tLqnKTZpjjuNuin.n...	User
198	198	user198	user198@example.com	\$2y\$10\$TC.7IXmWp6zP3RQ9aR61U0S...	User
199	199	user199	user199@example.com	\$2y\$10\$5qKSjs.zM2aKvsytZyJqh0E...	User
200	200	user200	user200@example.com	\$2y\$10\$BMSSdAaUjxiAKYpS.8fh6uL...	User
201	201	user201	user201@example.com	\$2y\$10\$HvTHke9/F6SiTmx3aXhk0eU...	User
202	202	user202	user202@example.com	\$2y\$10\$KeXTqSasW/0116Rq/Nqh200...	User
203	203	user203	user203@example.com	\$2y\$10\$cK5P/7Ijzf/m0ZXKQ2bp7eg...	User
204	204	user204	user204@example.com	\$2y\$10\$mA96SMvmc4Pj/9GsNfR4Te4...	User
205	205	user205	user205@example.com	\$2y\$10\$u.FJhcXHWs0urp6Ef22J3eT...	User
206	206	user206	user206@example.com	\$2y\$10\$TjhbdVI8X0Y.RlSrh3xWseo...	User
207	207	user207	user207@example.com	\$2y\$10\$QEbP0ri6u/h7Ed/ss1J/aeY...	User
208	208	user208	user208@example.com	\$2y\$10\$tj0TaYF8/Raoow1rNkcUA0d...	User
209	209	user209	user209@example.com	\$2y\$10\$uoB5y6RZnos3DhgSyLQjxeI...	User
210	210	user210	user210@example.com	\$2y\$10\$3PWBuVHS6RniDwiSveF5ueR...	User

Data snapshot for pets table (total 598 pets)

id	name	species	breed	color	photo_url	status	description	date_reported	user_id
1	Steam	Dog	Dalmatian	Tan	/images/pets/dogs/dog1.jpg	lost	discovered near the bridg...	2025-11-07 16:32:46	1
2	Dog2	Dog	Maltese	Black	/images/pets/dogs/dog2.jpg	found	Black Maltese Dog2 found ...	2025-11-07 16:32:46	1
3	Dog3	Dog	Dalmatian	Chocolate	/images/pets/dogs/dog3.jpg	lost	Chocolate Dalmatian Dog3 ...	2025-11-07 16:32:46	1
4	Dog4	Dog	Rottweiler	Black	/images/pets/dogs/dog4.jpg	found	Black Rottweiler Dog4, ve...	2025-11-07 16:32:46	1
5	Dog5	Dog	Dalmatian	Grey	/images/pets/dogs/dog5.jpg	lost	Grey Dalmatian Dog5, very...	2025-11-07 16:32:46	3
6	Dog6	Dog	Maltese	Tan	/images/pets/dogs/dog6.jpg	found	Tan Maltese Dog6 seen fo...	2025-11-07 16:32:46	3
7	Dog7	Dog	Pomeranian	Tan	/images/pets/dogs/dog7.jpg	lost	Tan Pomeranian Dog7 was p...	2025-11-07 16:32:46	4
8	Dog8	Dog	Border Collie	Golden	/images/pets/dogs/dog8.jpg	found	Golden Border Collie Dog8...	2025-11-07 16:32:46	4
9	Dog9	Dog	Dalmatian	Brindle	/images/pets/dogs/dog9.jpg	lost	Brindle Dalmatian Dog9 fo...	2025-11-07 16:32:46	5
10	Dog10	Dog	Poodle	Blue Merle	/images/pets/dogs/dog10.j...	found	Blue Merle Poodle Dog10, ...	2025-11-07 16:32:46	5
11	Dog11	Dog	Australian Shepherd	Cream	/images/pets/dogs/dog11.j...	lost	Cream Australian Shepherd...	2025-11-07 16:32:46	6
12	Dog12	Dog	Papillon	Black	/images/pets/dogs/dog12.j...	found	Black Papillon Dog12 disc...	2025-11-07 16:32:46	6
13	Dog13	Dog	Pomeranian	Fawn	/images/pets/dogs/dog13.j...	lost	Fawn Pomeranian Dog13, ve...	2025-11-07 16:32:46	7
14	Dog14	Dog	Bulldog	Brindle	/images/pets/dogs/dog14.j...	found	Brindle Bulldog Dog14 fou...	2025-11-07 16:32:46	7
15	Dog15	Dog	Maltese	Red	/images/pets/dogs/dog15.j...	lost	Red Maltese Dog15, very p...	2025-11-07 16:32:46	8
16	Dog16	Dog	Rottweiler	Black and White	/images/pets/dogs/dog16.j...	found	Black and White Rottweil...	2025-11-07 16:32:46	8
17	Dog17	Dog	Bichon Frise	White	/images/pets/dogs/dog17.j...	lost	White Bichon Frise Dog17 ...	2025-11-07 16:32:46	9
18	Dog18	Dog	Husky	Tan	/images/pets/dogs/dog18.j...	found	Tan Husky Dog18, very pla...	2025-11-07 16:32:46	9

id	name	species	breed	color	photo_url	status	description	date_reported	user_id
581	Bird181	Bird	Cockatiel	Olive	/images/pets/birds/bird181.jpg	lost	Olive Cockatiel Bird181 found near the park entrance.	2025-11-07 17:14:42	81
582	Bird182	Bird	Canary	Teal	/images/pets/birds/bird182.jpg	lost	Teal Canary Bird182, very friendly.	2025-11-07 17:14:42	82
583	Bird183	Bird	Budgerigar	Grey	/images/pets/birds/bird183.jpg	lost	Grey Budgerigar Bird183, seen near the fountain.	2025-11-07 17:14:42	83
584	Bird184	Bird	Cockatoo	Blue	/images/pets/birds/bird184.jpg	lost	Blue Cockatoo Bird184 found near the riverbank.	2025-11-07 17:14:42	84
585	Bird185	Bird	Peacock	White	/images/pets/birds/bird185.jpg	lost	White Peacock Bird185, very beautiful.	2025-11-07 17:14:42	85
586	Bird186	Bird	Toucan	White	/images/pets/birds/bird186.jpg	lost	White Toucan Bird186 found near the playground.	2025-11-07 17:14:42	86
587	Bird187	Bird	Swallow	Red	/images/pets/birds/bird187.jpg	lost	Red Swallow Bird187, very fast.	2025-11-07 17:14:42	87
588	Bird188	Bird	Macaw	Yellow	/images/pets/birds/bird188.jpg	lost	Yellow Macaw Bird188 was seen near the market.	2025-11-07 17:14:42	88
589	Bird189	Bird	Cockatiel	Grey	/images/pets/birds/bird189.jpg	lost	Grey Cockatiel Bird189, very calm.	2025-11-07 17:14:42	89
590	Bird190	Bird	Peacock	Olive	/images/pets/birds/bird190.jpg	lost	Olive Peacock Bird190 was seen near the garden.	2025-11-07 17:14:42	90
591	Bird191	Bird	Robin	Blue	/images/pets/birds/bird191.jpg	lost	Blue Robin Bird191 seen near the school.	2025-11-07 17:14:42	91
592	Bird192	Bird	Swallow	White	/images/pets/birds/bird192.jpg	lost	White Swallow Bird192 disappeared near the park.	2025-11-07 17:14:42	92
593	Bird193	Bird	Cockatiel	Crimson	/images/pets/birds/bird193.jpg	lost	Crimson Cockatiel Bird193, very colorful.	2025-11-07 17:14:42	93
594	Bird194	Bird	Peacock	White	/images/pets/birds/bird194.jpg	lost	White Peacock Bird194, very elegant.	2025-11-07 17:14:42	94
595	Bird195	Bird	Parakeet	White	/images/pets/birds/bird195.jpg	lost	White Parakeet Bird195, very lively.	2025-11-07 17:14:42	95
596	Bird196	Bird	Cockatoo	Orange	/images/pets/birds/bird196.jpg	lost	Orange Cockatoo Bird196, very friendly.	2025-11-07 17:14:42	96
597	Bird197	Bird	Budgerigar	Blue	/images/pets/birds/bird197.jpg	lost	Blue Budgerigar Bird197 found near the fountain.	2025-11-07 17:14:42	97
598	Bird198	Bird	Swallow	Yellow	/images/pets/birds/bird198.jpg	lost	Yellow Swallow Bird198 found near the riverbank.	2025-11-07 17:14:42	98
599	Bird199	Bird	Nightingale	Crimson	/images/pets/birds/bird199.jpg	lost	Crimson Nightingale Bird199, very melodious.	2025-11-07 17:14:42	99

Data snapshot for sightings table (total 153 sightings)

id	pet_id	user_id	comment	latitude	longitude	timestamp
2	1	1	Friendly dog seen sniffing around the park entrance.	-45.6789	123.4567	2025-06-03 09:45:21
4	3	2	Dog spotted running along the riverside path.	-23.4567	45.6789	2025-06-03 11:30:42
5	4	2	Playful pup seen chasing pigeons near the fountain.	56.789	-12.3456	2025-06-03 12:55:11
6	5	2	Large black dog resting under a shady tree.	-34.5678	67.8901	2025-06-04 08:22:51
7	6	3	Heard a dog barking near the playground fence.	23.4567	-56.789	2025-06-04 09:15:42
8	7	3	Dog seen walking calmly near the library steps.	-12.3456	34.5678	2025-06-04 10:48:30
9	8	3	Stray dog resting near the supermarket parking lot.	45.6789	-123.4567	2025-06-04 12:20:11
10	9	109	Golden retriever running happily by the riverbank.	-56.789	12.3456	2025-06-04 14:05:55
11	10	110	Dog seen waiting near the bus stop shelter.	10.1234	-70.5678	2025-06-05 08:18:42
12	11	111	Brown dog resting beside the bakery door.	-20.2345	60.6789	2025-06-05 09:52:13
13	12	112	Small puppy wandering around the playground grass.	35.3456	-90.789	2025-06-05 11:37:29
14	13	113	Dog spotted sitting near the park fountain.	-40.4567	75.8901	2025-06-05 12:44:17
15	14	114	White dog seen playing near the riverside trail.	50.5678	-100.1234	2025-06-05 13:29:50
16	15	115	Dog resting near the café garden area.	-15.6789	45.2345	2025-06-06 07:21:41

id	pet_id	user_id	comment	latitude	longitude	timestamp
159	435	195	Bird singing from the top of a lamppost.	-35.4567	45.789	2025-06-10 08:14:26
160	436	196	Bird fluttering near the park entrance.	40.5678	-90.1234	2025-06-10 09:52:44
161	437	197	Bird circling above the town fountain.	-10.6789	75.2345	2025-06-10 10:40:11
162	438	198	Bird seen landing on the bus stop railing.	55.789	-35.3456	2025-06-10 11:28:53
163	439	199	Bird resting beside the river stones.	-20.1234	80.4567	2025-06-10 12:55:22
164	440	200	Bird perched near the café garden fence.	30.2345	-100.5678	2025-06-11 08:11:49
165	441	201	Bird hopping between the flower pots at the plaza.	-25.3456	55.6789	2025-06-11 09:49:37
166	442	202	Bird gliding over the open field by the park.	45.4567	-120.789	2025-06-11 10:36:21
167	443	203	Bird flying around the fountain in circles.	-15.5678	65.1234	2025-06-11 11:22:58
168	444	204	Bird seen near the playground fence.	50.6789	-110.2345	2025-06-12 08:12:44
169	445	188	Bird perched near the riverside bench.	-35.789	40.3456	2025-06-12 09:41:22
170	446	189	Bird flying above the café garden.	20.1234	-75.4567	2025-06-12 11:18:59
171	447	190	Bird hopping along the path near the market.	-45.2345	85.5678	2025-06-12 12:55:47
172	448	191	Bird resting near the old bridge railing.	34.5678	-118.2345	2025-06-13 08:12:32
173	449	192	Bird perched on a signpost near the school.	-23.4567	45.6789	2025-06-13 09:45:21

Data snapshot for locations table (total 153 locations row)

id	pet_id	latitude	longitude	timestamp
1	2	-45.6789	123.4567	2025-06-03 09:45:21
2	3	34.5678	-98.7654	2025-06-03 10:17:55
3	4	-23.4567	45.6789	2025-06-03 11:30:42
4	5	56.789	-12.3456	2025-06-03 12:55:11
5	6	-34.5678	67.8901	2025-06-04 08:22:51
6	7	23.4567	-56.789	2025-06-04 09:15:42
7	8	-12.3456	34.5678	2025-06-04 10:48:30
8	9	45.6789	-123.4567	2025-06-04 12:20:11
9	10	-56.789	12.3456	2025-06-04 14:05:55
10	11	10.1234	-70.5678	2025-06-05 08:18:42
11	12	-20.2345	60.6789	2025-06-05 09:52:13
12	13	35.3456	-90.789	2025-06-05 11:37:29
13	14	-40.4567	75.8901	2025-06-05 12:44:17
14	15	50.5678	-100.1234	2025-06-05 13:29:50
15	16	-15.6789	45.2345	2025-06-06 07:21:41
16	17	60.789	-30.3456	2025-06-06 08:54:28
17	18	-25.1234	80.4567	2025-06-06 09:42:13

id	pet_id	latitude	longitude	timestamp
397	437	-10.6789	75.2345	2025-06-10 10:40:11
398	438	55.789	-35.3456	2025-06-10 11:28:53
399	439	-20.1234	80.4567	2025-06-10 12:55:22
400	440	30.2345	-100.5678	2025-06-11 08:11:49
401	441	-25.3456	55.6789	2025-06-11 09:49:37
402	442	45.4567	-120.789	2025-06-11 10:36:21
403	443	-15.5678	65.1234	2025-06-11 11:22:58
404	444	50.6789	-110.2345	2025-06-12 08:12:44
405	445	-35.789	40.3456	2025-06-12 09:41:22
406	446	20.1234	-75.4567	2025-06-12 11:18:59
407	447	-45.2345	85.5678	2025-06-12 12:55:47
408	448	34.5678	-118.2345	2025-06-13 08:12:32
409	449	-23.4567	45.6789	2025-06-13 09:45:21
410	450	25.2345	-88.4567	2025-06-13 11:28:12
411	517	44	44	2025-11-08 01:55:01
412	251	-33	-90	2025-11-08 02:00:00
413	502	23.999	-26	2025-11-08 02:17:00
414	410	44	88	2025-11-09 14:11:40
415	500	33.99	144.88	2025-11-09 14:47:21